FARM-TO-MARKET SUPPLY CHAIN PLATFORM

A Report

Submitted in partial fulfillment of the Requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERIG

Submitted By

CH. BHAVANI	22AJ1A0516
MVS MANIKANTA	22AJ1A0546
M. JYOTHI	22AJ1A0558
B. VENKAT CHOWDARY	22AJ1A0508

Under the Guidance of

MRS. BHAGYA LAKSHMI (M.Tech)

Assistant Professior

DEPARTMENT OF COMPUTER SCEINCE ENGINEERING

AMRITA SAI INSTITUTE OF SCIENCE & TECHNOLOGY

(AUTONOMOUS)

Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK, Kakinada,
Recognized by UGC under 2(f) & 12(B) of 1956 Act.,
ISO 9001:2015 Certified Institution, Accredited by NAAC "A" Grade,
Paritala, Kanchikacherla, Krishna Dist, Andhra Pradesh- 521180.
www.amritasai.org.in, Phone: 0866 2428399.

APRIL 2026

CERTIFICATE

Certified this is a Bonafide record of the thesis work entitled **"FARM-TO-MARKET SUPPLY CHAIN PLATFORM"**, done by **CH. BHAVANI (22AJ1A0516), MVS MANIKANTA (22AJ1A0546), M. JYOTHI (22AJ1A0558), B. VENKAT CHOWDARY (22AJ1A0508).** Submitted to the faculty of Computer Science Engineering, in partial fulfilment of the requirements for the Degree of BACHELOR OF TECHNOLOGY in **COMPUTER SCIENCE AND ENGINEERING** from Jawaharlal Nehru Technological University Kakinada.

PROJECT GUIDE	**Head Of Department**
MRS. BHAGYA LAKSHMI (M.Tech)	**Dr. P. CHIRANJEEVI Ph.D**
Assistant Professior	**PROFESSOR & HOD**

EXTERNAL EXAMINAR

DECLARATION

We hereby declare that the work, which is being presented in this dissertation entitled **"FARM-TO-MARKET SUPPLY CHAIN PLATFORM"**, submitted towards the partial fulfillment of requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** with specialization in **COMPUTER SCIENCE AND ENGINEERING** , is a record of our work carried out under the supervision of **MRS. BHAGYA LAKSHMI (M.Tech), Assistant Professor**, AMRITA SAI INSTITUTE OF SCIENCE AND TECHNOLOGY, Paritala.

I have not submitted the matter embodied in this dissertation in this report for the award of any other degree. Further, the technical details Furnished in the various chapters in this report are purely relevant to the above project and there is no deviation from the theoretical point of view for design, development and implementation.

CH. BHAVANI	**22AJ1A0516**
MVS MANIKANTA	**22AJ1A0546**
M. JYOTHI	**22AJ1A0558**
B. VENKAT CHOWDARY	**22AJ1A0508**

ACKNOWLEDGEMENT

It gives me immense pleasure to thank my guide **MRS. BHAGYA LAKSHMI (M.Tech), Assistant Professor**, CSE for having been a source of constant inspiration, precious guidance and generous assistance during the project work. I deem it as a privilege to have worked under her able guidance. Without his close monitoring and valuable suggestions, this work would not have taken this shape. I feel that his help is substitutable and unforgettable.

I take the privilege to record my thanks to **H.O.D of CSE Dr. P. CHIRANJEEVI Ph.D.** for his encouragement, cooperation and valuable suggestions to complete project in time.

I take the privilege to record my thanks to Directories od academics of college **Dr. P CHIRANJEEVI PhD for** his encouragement, cooperation and valuable suggestions to complete project in time.

I am very thankful to honorable Principal **Dr M. SASIDHAR** and I express my sincere gratitude and deep sense of respect for making me available with all the required assistance and for his support and inspiration to carry out this project in the Institute.

I am thankful to the teaching and non-teaching staff of CSE department for their direct as well as indirect help in my project.

I am elated to avail myself to this opportunity to express my deep sense of gratitude to my parents.

CH. BHAVANI	**22AJ1A0516**
MVS MANIKANTA	**22AJ1A0546**
M. JYOTHI	**22AJ1A0558**
B. VENKAT CHOWDARY	**22AJ1A0508**

ABSTRACT

Our final year project, "**Farm-to-Market Supply Chain Platform**," proposes a decentralized, blockchain-powered system designed to establish a direct, transparent, and equitable connection between small-scale farmers and end buyers. The platform aims to disrupt traditional, inefficient supply chains by eliminating the numerous intermediaries that currently capture most of the profit. At its core, the system leverages Ethereum smart contracts to automate transactions, enforce transparent pricing, and guarantee instant payments upon delivery verification. It integrates IoT sensors for real-time monitoring of product quality and conditions throughout the logistics process. By creating a trusted, peer-to-peer marketplace, our solution ensures that up to 80% of the final retail price reaches the farmers—a significant increase from the current 25-30%—while providing buyers with verifiable quality and origin data. The platform is architecturally built on a React-based frontend, a Node.js backend, and the Ethereum blockchain, forming a comprehensive technological stack for a modern, fair-trade agricultural ecosystem.

CHAPTER NO	CONTENTS	Page No

1	**INTRODUCTION**	2
	1.1 INTRODUCTION	2
	1.2 OBJECTIVE	2
	1.3 SCOPE OF THE PROJECT	3
2	**LITERATURE REVIEW**	6
3	**SYSTEM ANALYSIS**	8
	3.1 EXISTING SYSTEM	9
	3.2 PROPOSED SYSTEM	9
4	**FEASIBILITY STUDY**	12
	4.1 ECONOMIC FEASIBILITY	12
	4.2 OPERATIONAL FEASIBILITY	12
	4.3 TECHNICAL FEASABILITY	13
	4.4 LEGAL FEASABILITY	13
5	**STRUCTURE OF THE PROJECT**	15
6	**SYSTEM REQUIREMENTS SPECIFICATION**	17
	6.1 FUNCTIONAL REQUIREMENTS	17
	6.2 NON-FUNCTIONAL REQUIREMENTS	18
7	**SYSTEM DESIGN**	20
	7.1 SYSTEM SPECIFICATIONS	20
	7.2 UML DIAGRAM	21
	7.2.1 USE CASE DIAGRAM	22
	7.2.2 CLASS DIAGRAM	23
	7.2.3 SEQUENCE DIAGRAM	24
	7.2.4 ER DIAGRAM	25
	7.2.5 ACTIVITY DIAGRAM	26
	7.2.6 DATA FLOW DIAGRAM	27
	7.2.7 COMPONENT DIAGRAM	28
8	**METHODOLOGY**	30
	8.1 DESIGN AND ARCHITECTURE	30
	8.2 TOOLS AND TECHNOLOGIES USED	33

	8.3 ALGORITHMIC APPROACH	34
	8.4 DATA COLLECTION AND PREPROCESSING	35
9	**IMPLEMENTATION AND RESULT**	38
	9.1 IMPLEMENTATION DETAILS	38
	9.2 CODE SNIPPETS	40
	9.3 RESULT AND OUTCOME	43
	9.4 PERFORMANCE EVALUATION	47
	9.5 USER MANUAL	47
	9.6 CHALLENGES FACED AND THEIR SOLUTION	49
	9.7 ADDITIONAL TECHNICAL DETAILS	51
10	**SYSTEM TESTING**	54
	10.1 TYPES OF TESTING	54
	10.2 SYSTEM TESTING	61
11	**CONCLUSION**	63
12	**FUTURE SCOPE**	65
13	**REFERENCES**	67
	APPENDICES	68

LIST OF FIGURES

S.NO	FIGURE NO	FIGURE NAME	PAGE NO
1	7.2.1	USE CASE DIAGRAM	22
2	7.2.2	CLASS DIAGRAM	23
3	7.2.3	SEQUENCE DIAGRAM	24
4	7.2.4	ER DIAGRAM	25
5	7.2.5	ACTIVITY DIAGRAM	26
6	7.2.6	DATA FLOW DIAGRAM	27
7	7.2.7	COMPONENT DIAGRAM	28
8	8.1.2	BLOCK DIAGRAM	32

CHAPTER 1

INTRODUCTION

1.1 ABOUT THE PROJECT

The **Farm-to-Market Supply Chain Platform** is a blockchain-based agricultural marketplace developed to connect farmers directly with buyers in a secure and transparent environment. The system aims to modernize the traditional agricultural supply chain by introducing a digital platform that ensures efficiency, reliability, and trust among all stakeholders.

In the conventional supply chain, farmers often face challenges such as price manipulation, delayed payments, and dependency on intermediaries. These middlemen reduce farmers' profits and create a lack of transparency in transactions. This project addresses these issues by providing a decentralized system where farmers can directly list their products and interact with buyers.

The platform uses blockchain technology to record every transaction in a tamper-proof and immutable ledger. Key processes such as product listing, order placement, payment updates, and delivery tracking are securely stored. This ensures data integrity and prevents fraud, making the entire system more trustworthy and accountable.

One of the major advantages of the system is its ability to provide traceability. Buyers can track the journey of agricultural products from farm to market, ensuring quality and authenticity. At the same time, farmers benefit from direct access to customers, faster payments, and improved pricing without the involvement of intermediaries.

Overall, the project promotes a fair and efficient agricultural ecosystem by leveraging modern technologies. It enhances transparency, builds trust between farmers and buyers, and contributes to the development of a sustainable and farmer-centric supply chain system.

1.2 OBJECTIVE

1 **Develop a Blockchain-Powered Marketplace:** Build a decentralized agricultural marketplace using Ethereum smart contracts to ensure tamper-proof recording of supply chain events.

2 **Eliminate Intermediaries:** Create a direct farmer-to-buyer connection that reduces dependency on middlemen and enables farmers to receive up to 80% of the retail price.

3 **Ensure Product Traceability:** Implement end-to-end product tracking from farm to buyer with batch codes, harvest dates, and origin details stored on blockchain.

4 **Enable Secure and Flexible Payments:** Integrate Razorpay test-mode online payment and Cash on Delivery (COD) options with blockchain-linked payment status updates.

5 **Provide Role-Based Access Control:** Implement separate functionalities for farmers, buyers, and admin roles secured by JWT-based authentication.

6 **Implement Real-Time Order Tracking:** Develop a public order tracking module with GPS delivery simulation to monitor product movement during the logistics phase.

7 **Build an Admin Monitoring Dashboard:** Provide administrators with comprehensive dashboards to supervise users, products, orders, and platform activities.

8 **Automate Supply Chain Logic:** Use smart contract algorithms to automate batch code generation, order numbering, payment reference generation, and blockchain status mapping.

9 **Improve Market Trust and Accountability:** Provide buyers with verifiable product origin and quality data, improving confidence in the agricultural marketplace.

10 **Create a Scalable Platform:** Design the architecture to support future enhancements including live deployment, real GPS integration, mobile applications, and advanced analytics.

1.3 SCOPE OF PROJECT

1 **Blockchain-Verified Supply Chain:** Securely records product creation, order placement, payment updates, and delivery events on the Ethereum blockchain.

2 **Farmer Product Management:** Farmers can add, update, and manage agricultural products with batch codes, harvest dates, origin, quantity, and pricing details.

3 **Buyer Marketplace:** Buyers can browse products, add to cart, place orders, and choose between online payment or cash on delivery.

4 **Public Order Tracking:** Anyone can track order progress using an order number, providing maximum transparency without requiring login.

5 **GPS Delivery Simulation:** Simulates real-time product movement during the delivery phase to demonstrate logistics tracking capabilities.

6 **Admin Control Panel:** Centralized dashboard for monitoring users, products, orders, and payment activities across the entire platform.

7 **Multi-Role Authentication:** Secure JWT-based authentication provides role-specific dashboards and permissions for farmers, buyers, and admin.

8 **Foundation for Future Scale:** The platform can be extended to include live blockchain deployment, real IoT integration, AI-based price prediction, and mobile applications.

CHAPTER 2

LITERATURE REVIEW

1. **Feng Tian (2016)[1]:** Introduced a blockchain and RFID-based system for agri-food traceability, enabling secure tracking of product details like batch, origin, and handling across the supply chain.

2. **Caro et al. (2018)[2]:** Proposed the AgriBlockIoT architecture integrating IoT sensors with blockchain to automate monitoring of product movement and storage conditions.

3. **Kamilaris et al. (2019)[3]:** Identified multiple blockchain applications in agriculture, highlighting benefits such as transparency, trust, and traceability, along with challenges like scalability and energy usage.

4. **Zhao et al. (2019)[4]:** Analyzed key challenges including data privacy, regulatory compliance, scalability, and high implementation costs in blockchain-based agricultural systems.

5. **Shahid et al. (2020)[5]:** Developed a blockchain-based supply chain system using Ethereum smart contracts for automating payments, delivery verification, and quality assurance.

6. **Marchese & Tomarchio (2022)[6]:** Proposed a modular blockchain-based traceability system focusing on secure architecture and efficient product tracking.

7. **Yogarajan et al. (2023)[7]:** Conducted a systematic review highlighting smart contract automation and role-based access as key features for effective agricultural platforms.

8. **Rashid et al. (2024)[8]:** Developed a blockchain-based application improving farmer profitability and supply chain efficiency, supporting fair pricing models.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The traditional farm supply chain depends entirely on manual records, physical intermediaries, and verbal agreements for product movement and sales. Farmers sell produce to local agents or mandis, who pass it to wholesalers, then to distributors, and finally to retailers before it reaches the consumer. Each layer extracts a margin, leaving farmers with only 25-30% of the final sale price. This system also lacks any standardized mechanism for quality verification, product origin tracking, or transparent pricing.

Technical Analysis:

- **Lack of Transparency:** Product origin, harvest date, batch details, and handling history are often lost as produce moves through intermediaries. No unified ledger exists to verify the farm-to-fork journey.

- **Traceability Gaps:** Consumers and buyers have no reliable way to verify the authenticity, quality, or origin of agricultural products. Counterfeit or adulterated products can enter the supply chain undetected.

- **Fragmented Data:** Farmers, agents, wholesalers, and logistics providers maintain separate, non-interoperable records. Dispute resolution regarding payment or delivery status becomes extremely difficult.

- **Security Risks:** Traditional centralized databases are vulnerable to unauthorized alterations. There is no immutable proof of transactions, product listings, or quality certifications.

- **Delayed and Opaque Payments:** Farmers often wait weeks or months for payment after delivery. Cash on Delivery collections are tracked informally in spreadsheets, leading to frequent disputes.

- **No Digital Infrastructure:** Most small-scale farmers lack access to any digital marketplace. They are entirely dependent on local agents who control pricing information asymmetrically.

Limitations of the Existing System:

- Low profit margins for farmers due to multiple intermediaries
- Lack of trust between buyers and sellers
- Absence of real-time tracking and monitoring
- Inefficient manual processes and delays
- No standardized system for quality verification.

3.2 PROPOSED SYSTEM

The proposed system is a "local-first" agricultural marketplace that utilizes blockchain technology to create a transparent, immutable proof-of-concept for agricultural supply chain workflows. It replaces manual and intermediary-dependent processes with automated smart contract logic and cryptographically secured records

Features of the Proposed System:

1. **Farmer Module:** Farmers can add, update, and manage agricultural products with details including batch code, harvest date, origin, quantity, and price. Each product listing triggers a blockchain event.

2. **Buyer Module:** Buyers can browse products, add to cart, place orders with quantity selection, and choose between Razorpay online payment or Cash on Delivery (COD).

3. **Blockchain Integration:** Product creation, order placement, payment updates, and delivery status changes are recorded as immutable events on the Ethereum blockchain via Solidity smart contracts.

4. **Public Order Tracking:** Any user can track order progress using an order number without requiring login, providing maximum transparency in logistics.

5. **GPS Delivery Simulation:** The system simulates GPS-based product movement during the delivery phase to demonstrate real-time logistics tracking capability.

6. **Admin Monitoring Dashboard:** Administrators can supervise users, products, orders, and platform activities through a centralized, real-time dashboard.

7. **Flexible Payment Support:** Razorpay test-mode integration supports online UPI/card payments, while COD is also available for buyers without digital payment access.

8. **Role-Based Access Control:** Dedicated dashboards and permissions for farmer, buyer, and admin roles secured with JWT authentication.

Advantages of the Proposed System:

- Higher transparency through blockchain-recorded supply chain events
- Verifiable product origin and quality data improves buyer trust
- Farmers receive fair pricing without intermediary deductions
- Automated smart contracts reduce manual errors and disputes
- Real-time order tracking provides visibility to all stakeholders
- Scalable architecture supports future production-grade deployment

CHAPTER 4

FEASIBILITY STUDY

A feasibility study is a critical aspect of any project, determining its viability and potential for success before committing full development resources. The following sections examine the feasibility of the Farm-to-Market Supply Chain Platform from economic, operational, technical, and legal perspectives.

4.1 ECONOMIC FEASIBILITY

- Low Development Cost: The project uses entirely open-source technologies — React.js, Node.js, MongoDB, Truffle, Ganache, and Web3.js — with no major licensing costs.
- Free Blockchain Testing Environment: Ganache provides a free local Ethereum blockchain for development and testing, eliminating the cost of testnet ETH for academic demonstrations.
- Affordable Deployment: Cloud deployment on Heroku or Railway costs approximately $0-$25/month at academic scale, making it financially accessible.
- Potential for Social Impact Monetization: The platform can generate revenue through transaction fees, premium analytics for buyers, or B2B licensing to agricultural cooperatives.
- Cost-Effective Maintenance: The modular architecture and well-documented codebase minimize ongoing maintenance costs and allow incremental feature additions.

4.2 OPERATIONAL FEASIBILITY

- User-Friendly Interface: The React.js frontend provides intuitive dashboards for all three user roles. Farmers with basic smartphone literacy can list products without technical training.
- Automated Workflows: Order processing, payment verification, and blockchain event recording are automated, eliminating manual data entry for routine operations.
- Multi-Language Potential: The platform can be extended with regional language support (Telugu, Hindi) to improve accessibility for rural farmers in Andhra Pradesh.
- Minimal Infrastructure Required: Farmers need only a smartphone and internet connectivity to access the full platform functionality.

4.3 TECHNICAL FEASIBILITY

- Proven Technology Stack: React.js, Node.js, Express.js, and MongoDB are mature, widely-adopted technologies with extensive documentation and community support.
- Blockchain Feasibility: Solidity smart contracts on Ethereum are an established standard for decentralized applications. Truffle and Ganache provide a complete local development environment.
- Scalability Path: The system can migrate from Ganache local blockchain to Sepolia testnet or Polygon mainnet for production deployment without architectural changes.
- Standard Hardware Requirements: The platform runs on a standard computer with Intel i3 or above and 4GB+ RAM, making it accessible for academic and small-business deployment.

4.4 LEGAL FEASIBILITY

- Compliance with E-Commerce Regulations: The platform operates as a marketplace facilitating direct buyer-seller transactions and does not engage in direct commodity trading that would require SEBI or APMC regulation.
- Data Privacy Compliance: MongoDB stores only user-provided data. No biometric or Aadhaar-linked data is collected. GDPR and IT Act 2000 principles are followed.
- Disclaimer for Predictions: The platform does not provide financial advice or price guarantees. A legal disclaimer clarifies the informational nature of all displayed pricing data.
- Smart Contract Transparency: All smart contract code is open-source and publicly auditable, ensuring no hidden terms or deceptive transaction logic.

CHAPTER 5

STRUCTURE OF THE PROJECT

The Software Development Life Cycle (SDLC) for this project follows an iterative and modular approach, ensuring each phase builds upon validated outputs of the previous stage.

1. **Planning**: Project requirements defined. Technology stack selected (React, Node.js, MongoDB, Ethereum). Monorepo structure established with /client, /server, and /blockchain directories.

2. **Analysis**: Detailed analysis of traditional supply chain problems conducted. Stakeholder needs for farmers, buyers, and admin roles identified. Blockchain use cases mapped to supply chain events.

3. **Design**: Three-tier architecture designed (Presentation, Business Logic, Data Layer). UML diagrams created including Use Case, Class, Sequence, ER, Activity, DFD, and Component diagrams.

4. **Development**: One-file-at-a-time implementation approach adopted for maintainability. Smart contracts written in Solidity and deployed via Truffle. Frontend built with React/Vite. Backend APIs implemented in Express.js. MongoDB schemas designed for users, products, orders, and delivery data.

5. **Testing**: Unit tests implemented for smart contract functions, API endpoints, and authentication logic. Integration testing performed across frontend-backend-blockchain layers. System testing validated end-to-end user flows.

6. **Deployment**: Local deployment using Ganache blockchain and MongoDB localhost. Application served via Node.js development server. Future deployment path: Railway/Heroku for backend, Vercel for frontend, Sepolia for blockchain.

7. **Maintenance**: Modular code structure allows independent updates to each layer. GitHub version control ensures traceability of all changes. Documentation maintained alongside source code.

The SDLC for this project may be iterative, meaning that the team will repeat some or all of these phases as needed to ensure that the software meets the project's goals and objectives.

CHAPTER 6

SYSTEM REQUIREMENTS SPECIFICATION

6.1 FUNCTIONAL REQUIREMENTS

1. **User Registration & Authentication:** Farmers, buyers, and admins can register, login, and logout securely. JWT tokens manage session and role-based access control.

2. **Product Management**: Farmers can create, update, and delete product listings with fields for name, description, batch code, harvest date, origin, quantity, price, and images.

3. **Marketplace Browsing:** Buyers can browse, search, and filter available products by category, price range, origin, and availability.

4. **Cart and Order Placement:** Buyers can add products to cart, adjust quantities, and place orders with delivery address and payment method selection.

5. **Blockchain Event Recording:** Product creation, order placement, payment status updates, and delivery confirmations are recorded as immutable events on the Ethereum blockchain.

6. **Payment Processing:** Razorpay test-mode integration enables online UPI/card/net banking payments. COD option also available for offline payment.

7. **Order Tracking:** Public tracking module allows any user to check order status, delivery updates, and supply chain event history using an order number.

8. **GPS Delivery Simulation:** Delivery movement is simulated with GPS coordinates updating at intervals to demonstrate real-time logistics tracking.

9. **Admin Dashboard:** Admin can view all users, products, orders, and platform statistics. Admin can deactivate users or resolve flagged orders.

10. **Notifications:** Users receive status updates when orders are confirmed, dispatched, out for delivery, and delivered.

6.2 NON-FUNCTIONAL REQUIREMENTS

1. Performance: API responses should complete within 2 seconds for standard queries. Blockchain transaction confirmation should complete within Ganache block time for local testing.

2. Security: JWT authentication with token expiry. Passwords hashed using bcrypt with appropriate salt rounds. API routes protected with middleware authentication checks.

3. Scalability: Monorepo architecture supports independent scaling of frontend, backend, and blockchain layers. MongoDB sharding can be enabled for database scaling.

4. Reliability: Error handling middleware in Express.js prevents application crashes. Smart contract validation prevents invalid state transitions.

5. Usability: Role-specific dashboards reduce cognitive load. Public order tracking requires no login. Mobile-responsive React UI accessible on smartphones.

6. Maintainability: Modular code structure separates concerns. Environment variables (.env) manage configuration. GitHub version control tracks all changes.

7. Legal Compliance: No personal data shared with third parties without consent. Razorpay test mode ensures no actual financial transactions during academic demonstration.

CHAPTER 7

SYSTEM DESIGN

7.1 SYSTEM SPECIFICATIONS

HARDWARE / SOFTWARE	SPECIFICATION
PROCESSOR	Intel i3 or above (64-bit)
RAM	Minimum 4 GB (8 GB Recommended)
STORAGE	Minimum 20 GB free disk space
INTERNET	Required for package installation and payment API testing
OPERATING SYSTEM	Windows 10/11, Linux, or macOS
FRONTEND	React.js + Vite
BACKEND	Node.js + Express.js
DATABASE	MongoDB
BLOCKCHAIN	Solidity, Truffle, Ganache, Web3.js
LANGUAGES	JavaScript, Solidity, HTML, CSS
AUTHENTICATION	JWT (JSON Web Tokens)
PAYMENT GATEWAY	Razorpay Test Mode
IDE	Visual Studio Code
VERSION CONTROL	Git and GitHub

7.2 UML DIAGRAMS

UML (Unified Modeling Language) diagrams are a visual representation of the software system or application being developed. They are used to describe, specify, design, and document the various aspects of the system under development. UML diagrams provide a standardized way of modeling software systems, making it easier for developers, analysts, and stakeholders to understand and communicate the system's design. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML. The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

- The Primary goals in the design of the UML are as follows:
- Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts. Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modelling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

7.2.1 USE CASE DIAGRAM

A use case diagram is a type of UML diagram that represents the interactions between users and a system. It is a graphical representation of a system's functionality and the actors (users) that interact with it. The main purpose of a use case diagram is to define the system's requirements, which helps developers understand how the system will be used and what features it should have.

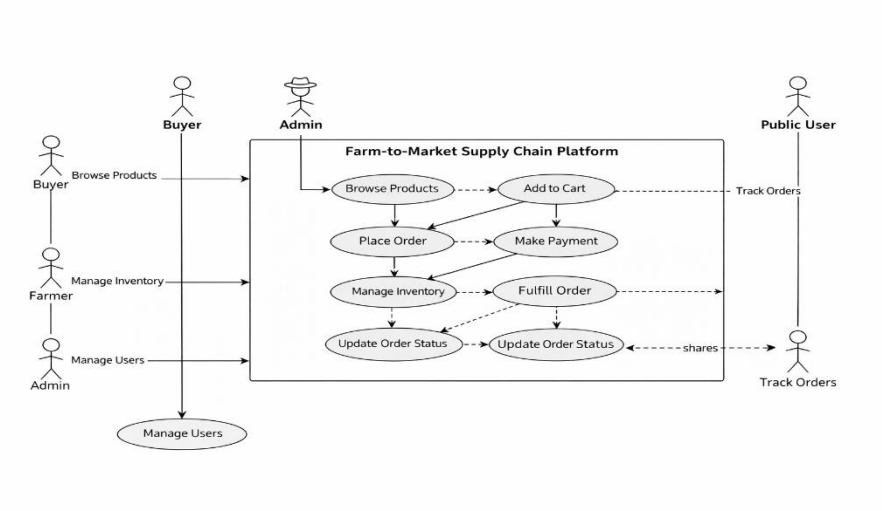


Fig 7.2.1 Use Case Diagram

In a use case diagram, the actors are represented by stick figures, and the system's functionality is represented by ovals or rectangles. Lines with arrows connect the actors to the system's functionality, indicating the actions they can perform within the system.

Use case diagrams are useful in many ways. They help to:

1. Define the system's requirements
2. Visualize the interactions between the system and its users
3. Identify the different types of users and their roles
4. Identify the system's functionality
5. Analyse the system's behaviour and identify potential issues.

Overall, a use case diagram is a powerful tool for analyzing, designing, and documenting systems. It provides a clear and concise view of the system's functionality and how users interact with it, making it an essential part of the software development process.

7.2.2 CLASS DIAGRAM

A class diagram is a type of UML diagram that represents the structure of a system by showing the classes and their relationships to one another. Classes are the building blocks of object-oriented programming, and a class diagram provides a graphical representation of these classes.

The primary elements of a class diagram are classes, interfaces, associations, and attributes. A class is represented by a rectangle with the class name inside. Attributes are properties of the class that are represented as smaller rectangles connected to the class rectangle. Associations are represented as lines between classes, with optional arrowheads indicating the direction of the relationship.

In addition to these basic elements, class diagrams can also show inheritance, aggregation, and composition. Inheritance is represented by a line with a triangular arrowhead pointing from the subclass to the superclass. Aggregation and composition are both types of association that represent a whole-part relationship between classes. Aggregation is represented by a line with an open diamond on the part end, while composition is represented by a line with a filled diamond on the part end. Overall, class diagrams provide a clear and concise way to represent the structure of a system and its components, making it easier to understand and communicate the system's design.

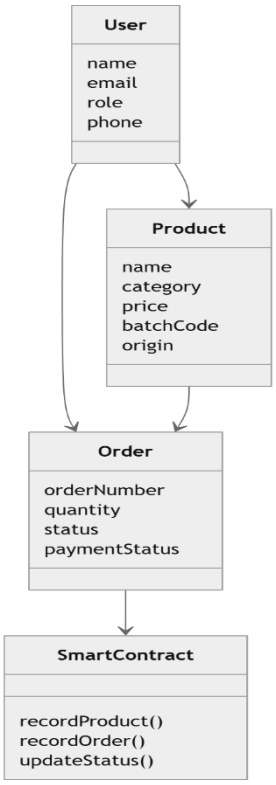


Fig 7.2.2 Class Diagram

7.2.3 SEQUENCE DIAGRAM

A sequence diagram is a type of UML diagram that shows interactions between objects or components in a system in a time-based sequence. It illustrates how objects exchange messages and interact with each other to perform a specific task or function.

In a sequence diagram, time flows from top to bottom, with the vertical axis representing time and the horizontal axis representing objects or components. The diagram consists of objects or actors, lifelines, messages, and activation bars.

- **Objects or Actors:** Objects are represented as boxes, and actors are represented as stick figures. They represent the entities involved in the system.
- **Lifelines:** Lifelines are vertical dashed lines that represent the lifespan of an object or actor during the interaction.
- **Messages:** Messages are represented by arrows and show the communication or interaction between objects or actors. They can be synchronous or asynchronous, and they are labelled with the operation or method being performed.
- **Activation bars:** Activation bars represent the duration of time that an object or actor is busy processing a request.

Sequence diagrams are useful for visualizing the interactions and flow of control in a system and are commonly used in software development to model and design the behavior of a system.

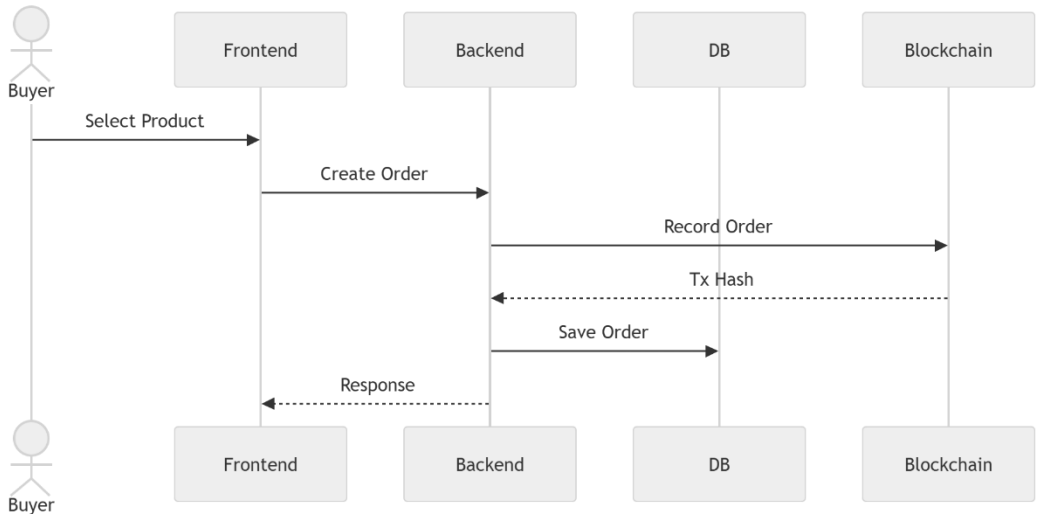


Fig 7.2.3 Sequence Diagram

7.2.4 ER DIAGRAM

An Entity-Relationship (ER) diagram is a graphical representation of entities and their relationships to each other. The entities can be any object, person, place, or thing about which data can be stored, and the relationships can be any connection or association between those entities.

ER diagrams consist of entities represented as rectangles, attributes represented as ovals or circles, and relationships represented as diamonds. The entities are connected to the attributes using lines, and relationships are connected to the entities using lines with arrows.

The ER diagram is a useful tool in database design as it helps to visualize the relationships between entities and to identify the attributes required for each entity. It also helps in identifying the type of relationships between entities, such as one-to-one, one-to-many, or many-to-many.

Overall, ER diagrams are an important part of database design and development, helping to ensure the accuracy, efficiency, and effectiveness of data storage and retrieval.

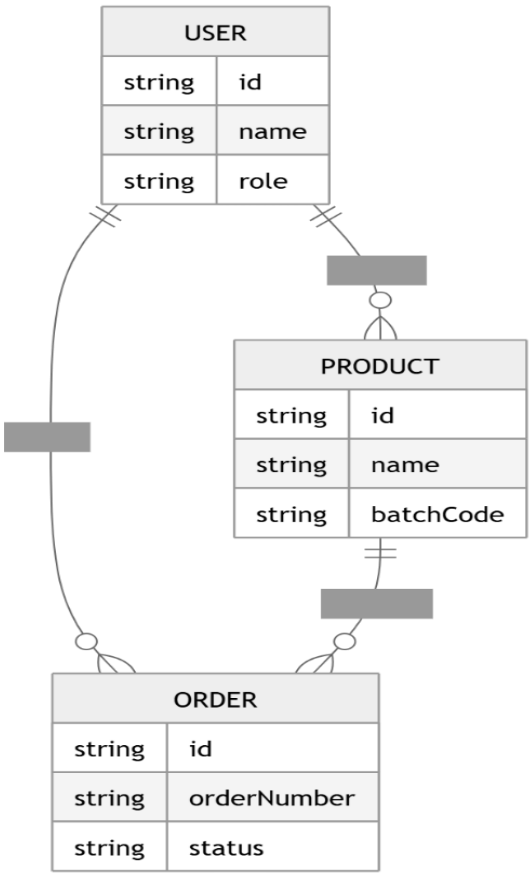


Fig 7.2.4 ER Diagram

7.2.5 ACTIVITY DIAGRAM

An activity diagram is a type of UML diagram that is used to visualize the flow of activities, actions, and decision points in a system or process. It is a graphical representation of workflows or processes, showing the steps as nodes or boxes and the transitions between them as arrows or lines.

Activity diagrams are often used to model complex business processes or software systems, as they provide a high-level view of the system's behavior and help identify potential issues or bottlenecks in the process.

In an activity diagram, the nodes or boxes represent the activities or actions in the process, while the arrows or lines represent the transitions or decisions between them. Activities can be sequential, parallel, or conditional, and decision points can be used to branch the flow of the process based on certain conditions or criteria.

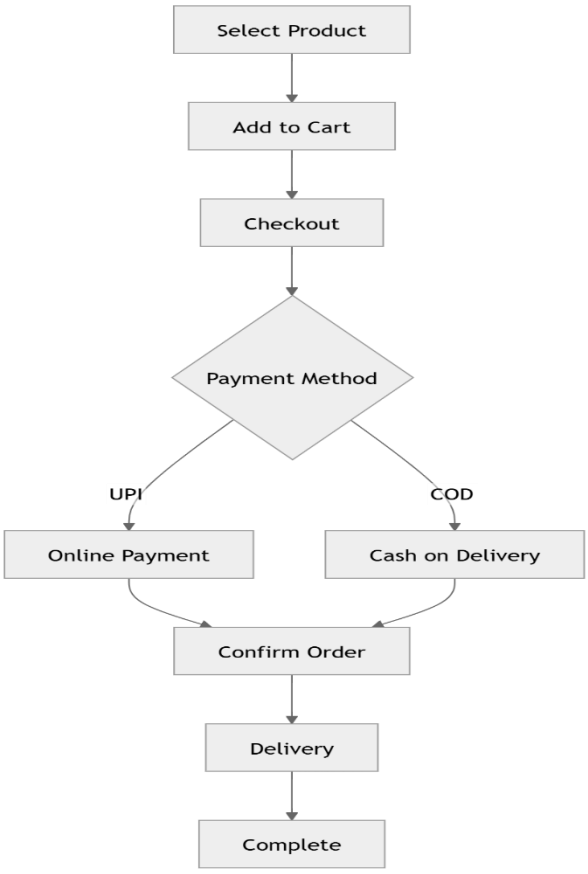


Fig 7.2.5 Activity Diagram

7.2.6 DATA FLOW DIAGRAM

A data flow diagram (DFD) is a visual representation of how data flows through a system or organization. It uses symbols and arrows to depict the flow of information between processes, data sources, and users.

DFDs are commonly used in software engineering, systems engineering, and business process management to help understand, analyze, and improve processes.

The symbols used in a DFD are:

1. Process: a rectangle with rounded corners, which represents a transformation of data
2. Data source or sink: a rectangle with no rounded corners, which represents a source or destination of data
3. Data flow: an arrow, which represents the movement of data between processes and data sources/sinks
4. External entity: a rectangle with a double line border, which represents an external system or organization that interacts with the system being modelled
5. A DFD typically has multiple levels, where each level provides increasing detail about the processes and data flows of the system being modeled. The highest level DFD represents the entire system, while lower level DFDs provide more detailed views of individual processes.
6. Overall, a DFD is a useful tool for understanding the flow of data in a system or organization, and can be used to identify potential bottlenecks, inefficiencies, or opportunities for improvement.

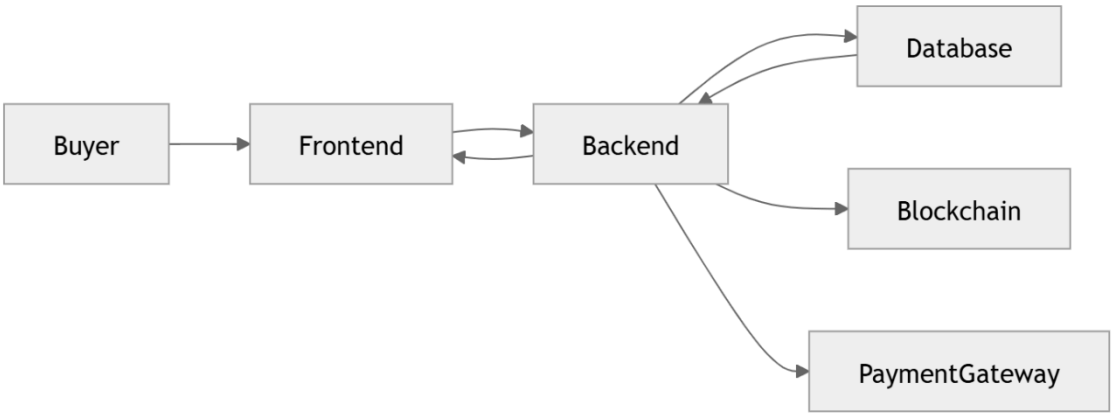


Fig 7.2.6 Data Flow Diagram

7.2.7 COMPONENT DIAGRAM

A component diagram is a type of UML diagram that shows the organization and dependencies of a system's components, including their interfaces and interactions. It is used to visualize the high- level architecture of a system and to communicate the relationships between components to stakeholders.

Components are represented as rectangles with optional ports, which are points of interaction between the component and its environment. Interfaces, which define the operations and services provided by a component, are shown as labeled connectors between components. Dependencies between components are depicted with directed arrows.

Component diagrams can be used in various stages of software development, from system design to implementation and maintenance. They help developers to better understand the overall structure of a system and to identify potential issues with the component interactions.

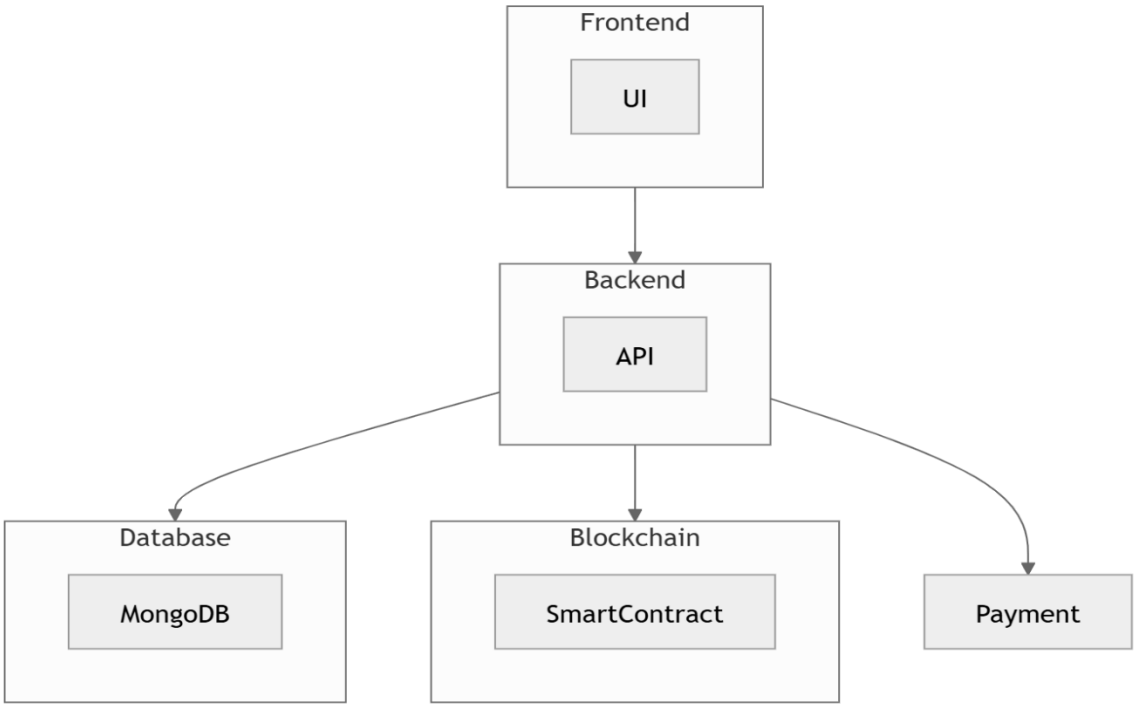


Fig 7.2.7 Component Diagram

CHAPTER 8

METHODOLOGY

The methodology of the **Farm-to-Market Supply Chain Platform** focuses on designing and implementing a blockchain-based agricultural marketplace that ensures transparency, traceability, and secure transactions. The system follows a structured approach combining web technologies with blockchain integration.

8.1 DESIGN AND ARCHITECTURE

The **Farm-to-Market Supply Chain Platform** is designed using a modular and scalable architecture that integrates modern web technologies with blockchain to ensure transparency, security, and efficiency in agricultural supply chain management. The system follows a **three-tier architecture**, consisting of the presentation layer, application layer, and data & blockchain layer.

1. System Architecture Overview

The system is designed using a **three-tier architecture**, consisting of:

The Farm-to-Market Supply Chain Platform follows a multi-layered architecture that integrates frontend, backend, database, and blockchain components to ensure transparency, scalability, and security. The system is designed to efficiently handle user interactions, data processing, and immutable record storage.

2. Architecture Components

The system consists of four major components:

A. **Frontend (Presentation Layer):** Built using React.js, this layer provides a user-friendly interface for buyers, farmers, administrators, and public users. It handles user interactions such as product browsing, order placement, and tracking.

B. **Backend (Application Layer):** Developed using Node.js and Express.js, this layer manages business logic, API requests, authentication, and communication between frontend, database, and blockchain.

C. **Database (Data Layer):** MongoDB is used to store user data, product details, orders, and transaction history. It ensures fast and flexible data management.

D. **Blockchain (Trust Layer):** Ethereum smart contracts are used to store critical supply chain events such as product creation, order placement, and payment verification. This ensures tamper-proof and transparent records.

3. System Workflow

1. The user interacts with the frontend interface (login, browse, order).
2. The frontend sends requests to the backend APIs.
3. The backend processes the request and:
    - Stores/retrieves data from MongoDB
    - Records important events on the blockchain
4. Payment processing is handled via Razorpay test mode.
5. The backend sends the response back to the frontend.
6. Users can track orders using real-time updates and blockchain references

4. System Design Considerations

- **Scalability:** Supports future expansion (mobile apps, cloud deployment)
- **Security:** JWT authentication + blockchain immutability
- **Transparency:** All major events recorded on blockchain
- **Efficiency:** Fast data handling using MongoDB
- **Modularity:** Each layer works independently.

5. Technology Stack

Technology Stack

Category	Technology / Tool
Frontend	React.js, Vite, HTML, CSS, JavaScript, Axios
Backend	Node.js, Express.js, JWT, bcrypt.js
Database	MongoDB, Mongoose
Blockchain	Solidity, Ethereum, Truffle, Ganache, Web3.js
Payment	Razorpay (Test Mode), COD
Development Tools	Visual Studio Code, Git, GitHub, Postman
Additional Tools	Recharts, Leaflet, CORS, Morgan

6. CONCLUSION

The **Farm-to-Market Supply Chain Platform** demonstrates an effective use of blockchain technology to improve transparency, traceability, and efficiency in agricultural supply chains. It enables direct interaction between farmers and buyers, reducing intermediaries and increasing farmer profits.

By integrating modern web technologies with Ethereum smart contracts, the system ensures secure transactions, real-time tracking, and tamper-proof record management. Overall, the project provides a scalable and reliable solution, with potential for future enhancements such as real-time deployment and mobile integration.

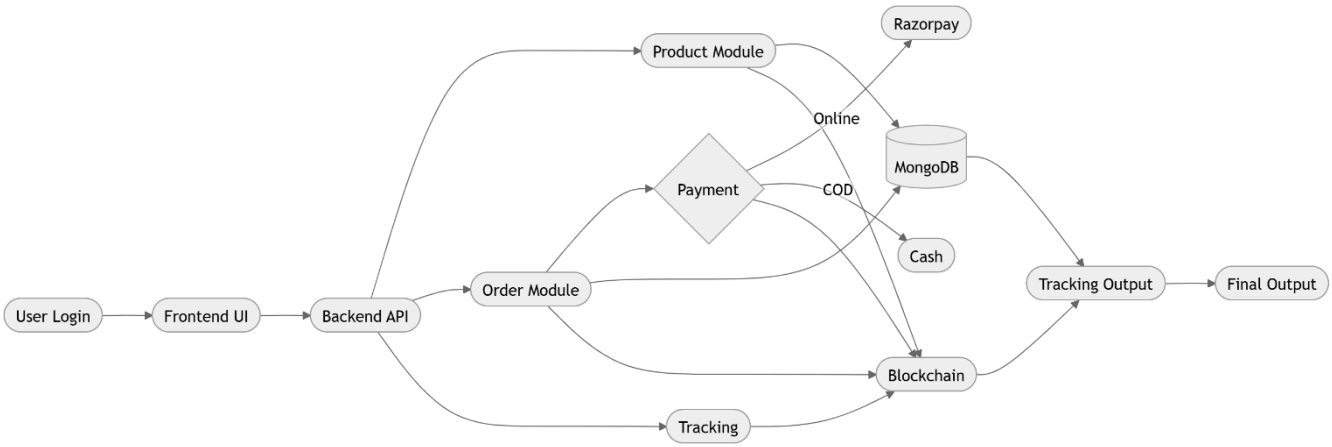


Fig 8.1.2 BLOCK DIAGRAM

8.2 TOOLS AND TECHNOLOGIES USED

The tools and technologies used in this project are selected based on the requirements of building a blockchain-based agricultural marketplace. These technologies ensure scalability, security, and efficient system performance.

Programming Languages: JavaScript is used for both frontend and backend development, while Solidity is used for writing smart contracts. HTML and CSS are used for designing the user interface.

Frameworks and Libraries: React.js and Vite are used for frontend development to create a responsive user interface. Node.js and Express.js are used for backend development to handle APIs and business logic.

Blockchain Technologies: Ethereum is used as the blockchain platform. Solidity is used to develop smart contracts, while Truffle and Ganache are used for development and testing. Web3.js is used to interact with the blockchain.

Database Management System: MongoDB is used as the database for storing user data, products, and orders. Mongoose is used for schema modeling and database interaction.

API Development and Testing: REST APIs are developed using Express.js. Tools like Postman are used for API testing and debugging.

Development Environment: Visual Studio Code is used as the primary IDE for coding, debugging, and development.

Version Control: Git and GitHub are used for source code management, collaboration, and version tracking.

Payment Integration: Razorpay (test mode) is used for online payment processing, along with Cash on Delivery (COD) support.

Additional Tools: Recharts is used for data visualization, and Leaflet is used for map-based delivery tracking.

8.3 ALGORITHM APPROACH

1. Password Hashing (bcrypt): User passwords are hashed using bcrypt with salt rounds before storing in MongoDB. During login, the submitted password is compared against the stored hash using bcrypt.compare(). This ensures that even if the database is compromised, plaintext passwords cannot be recovered.

2. JWT Authentication: Upon successful login, the backend generates a JSON Web Token signed with a secret key, containing userId, role, and expiry (expiresIn: "7d"). All protected API routes use an authMiddleware that verifies the token from the Authorization header and attaches the decoded user to req.user before proceeding.

3. Order Number Generation: A sequential order number is generated using the format ORD-XXXX where XXXX is zero-padded using the MongoDB document count for the orders collection. Example: if 42 orders exist, the next order number is ORD-0043. This ensures uniqueness and human-readable order references.

4. Batch Code Generation: Product batch codes are generated using a three-part composite format: first three characters of the product name (uppercase) + current date in DDMMYYYY format + a 4-character random alphanumeric suffix. Example: TOM27032025A3X7. This ensures each product batch has a unique, traceable identifier.

5. Payment Reference Generation: Unique payment reference IDs are constructed from the payment mode prefix (ONLINE_ or COD_) + order number + Unix timestamp + retry count suffix. This provides a deterministic, non-colliding reference for every payment attempt that can be correlated across MongoDB and blockchain records.

6. Blockchain Status Mapping: Payment mode (ONLINE/COD), payment status (PENDING/SUCCESS/FAILED), and order status (PLACED/DISPATCHED/DELIVERED) are mapped to smart contract enum integer values before calling the Solidity function. The mapping ensures type safety at the Solidity ABI boundary and prevents invalid state transitions.

8.4 DATA COLLECTION AND PREPROCESSING

The **Farm-to-Market Supply Chain Platform** collects and processes data from various user inputs and system-generated sources to ensure consistency, accuracy, and efficient system performance. Proper preprocessing is essential to maintain data integrity across the database and blockchain.

1. Data Collection

Data is collected from multiple sources within the system:

- **User Data:** Registration details such as name, email, phone number, and address
- **Product Data:** Product name, category, price, quantity, batch code, origin, and harvest date
- **Order Data:** Order number, product details, buyer and farmer information
- **Payment Data:** Payment method (UPI/COD), transaction details, payment status
- **Tracking Data:** Order status updates and GPS-based delivery location

2. Data Cleaning

- Remove incomplete or invalid inputs
- Validate required fields (price, quantity, contact details)
- Eliminate duplicate records
- Ensure correct data formats (dates, numbers, strings)

3. Data Transformation

- Convert price into **paise (integer format)** for accuracy
- Generate unique identifiers such as **batch codes** and **order numbers**
- Standardize address fields (location, state, pincode)
- Format timestamps for consistent storage

4. Data Integration

- Combine data from frontend forms, backend APIs, and blockchain responses
- Synchronize data between **MongoDB** and **blockchain records**
- Maintain consistent references such as transaction hashes and IDs

5. Data Storage

- Store processed data in **MongoDB collections** (Users, Products, Orders)
- Store critical events on the **blockchain** for immutability
- Maintain relationships using unique IDs

6. Data Validation

- Ensure product availability before order creation
- Verify payment details during transactions
- Authenticate users using JWT tokens
- Validate role-based access (Buyer, Farmer, Admin)

7. Data Pipeline Workflow

1. Collect data from user inputs and system events
2. Clean and validate the data
3. Transform data into standardized formats
4. Store data in MongoDB
5. Record key events on blockchain
6. Retrieve and display processed data to users

CHAPTER 9

IMPLEMENTATION AND RESULT

9.1 IMPLEMENTATION DETAILS

The implementation of the **Farm-to-Market Supply Chain Platform** follows a structured approach to integrate web technologies with blockchain for secure and transparent agricultural transactions. The system is divided into multiple stages to ensure efficient functionality.

1. User Authentication and Registration

Users register as Buyer, Farmer, or Admin. Secure login is implemented using JWT authentication, and passwords are encrypted using bcrypt. Role-based access control is enforced.

2. Product Management

Farmers can add, update, and manage products with details such as name, price, quantity, batch code, origin, and harvest date. Unique batch codes are generated automatically, and product data is stored in MongoDB and recorded on blockchain.

3. Order Management

Buyers can browse products, add items to cart, and place orders. The system validates product availability, generates unique order numbers, and stores order details in the database while recording them on blockchain.

4. Payment Processing

The system supports Razorpay test mode for online payments and Cash on Delivery (COD). Payment status is verified and updated, and transactions are recorded on blockchain.

5. Blockchain Implementation

Smart contracts written in Solidity record key events such as product creation, order placement, payment updates, and order status changes. Web3.js is used for backend interaction, and transaction hashes are stored for verification.

6. Order Tracking System

Users can track orders using order numbers. The system displays order status, history, GPS-based delivery simulation, and blockchain proof data.

7. Admin Dashboard

Admins manage users, products, and orders through a dashboard with monitoring and analytics features.

8. System Integration

The frontend communicates with the backend via REST APIs. The backend interacts with MongoDB and blockchain, and integrates payment services.

9. Deployment

The system runs in a local environment using Ganache for blockchain and can be extended for cloud deployment with secure configurations.

9.2 CODE SNIPPETS

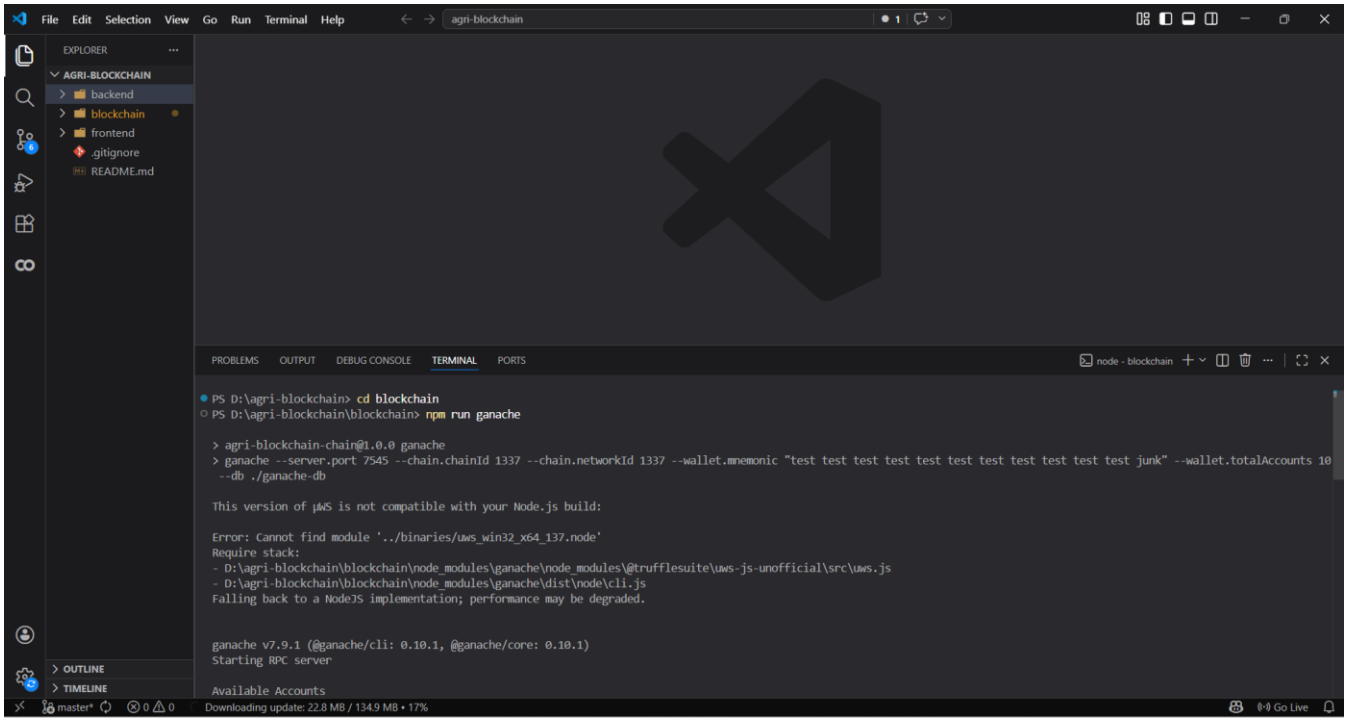
```
function recordOrder(
    string memory orderNumber,
    uint256 productId,
    uint256 quantity,
    uint256 totalPaise,
    PaymentMethod paymentMethod,
    PaymentStatus paymentStatus
) external onlyOwner returns (uint256) {
    Product storage product = products[productId];
    require(product.id != 0, "Product not found");
    require(product.isActive, "Product is inactive");
    require(product.quantity >= quantity, "Insufficient quantity");

    product.quantity -= quantity;
    orderCount += 1;

    orders[orderCount] = Order({
        id: orderCount,
        orderNumber: orderNumber,
        productId: productId,
        quantity: quantity,
        totalPaise: totalPaise,
        paymentMethod: paymentMethod,
        paymentStatus: paymentStatus,
        orderStatus: OrderStatus.Pending
    });

    emit OrderRecorded(orderCount, orderNumber, productId, quantity, totalPaise, paymentMethod);
    return orderCount;
}
```

```
async function recordOrder(orderInput) {
  const contract = getContract();
  const receipt = await sendContractTransaction(
    contract.methods.recordOrder(
      orderInput.orderNumber,
      Number(orderInput.productBlockchainId),
      Number(orderInput.quantity),
      Number(orderInput.totalPaise),
      toPaymentMethodEnum(orderInput.paymentMethod),
      toPaymentStatusEnum(orderInput.paymentStatus)
    )
  );

  const event = getOrderFromReceipt(receipt);

  return {
    receipt,
    blockchainOrderId: Number(event.orderId),
  };
}
```

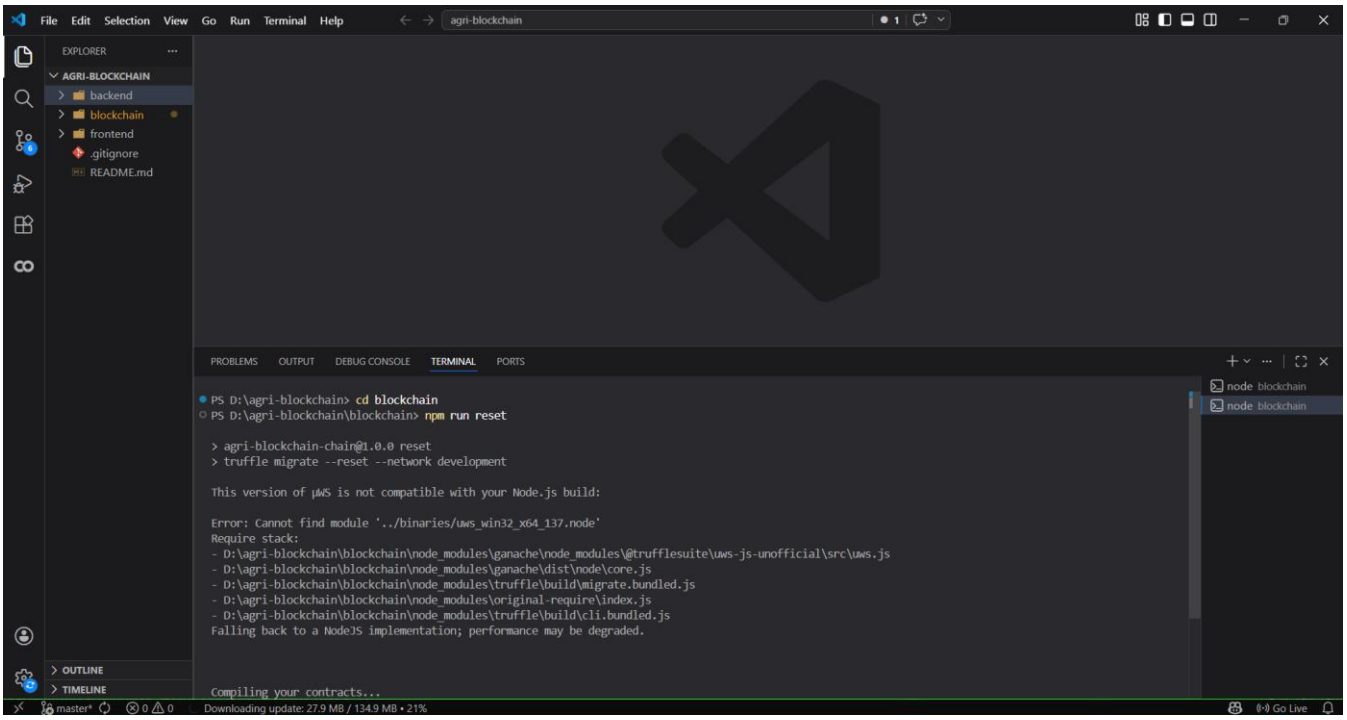
```javascript
async function simulateGpsMovement() {
  const orders = await Order.find({ status: "out_for_delivery" });

  for (let order of orders) {
    if (!order.location) {
      order.location = { lat: 21.1458, lng: 79.0882 };
    }

    const latChange = (Math.random() - 0.5) * 0.001;
    const lngChange = (Math.random() - 0.5) * 0.001;

    order.location.lat += latChange;
    order.location.lng += lngChange;

    await Order.updateOne(
      { _id: order._id },
      { $set: { location: order.location } }
    );
  }
}
```

9.3 RESULT AND OUTCOME

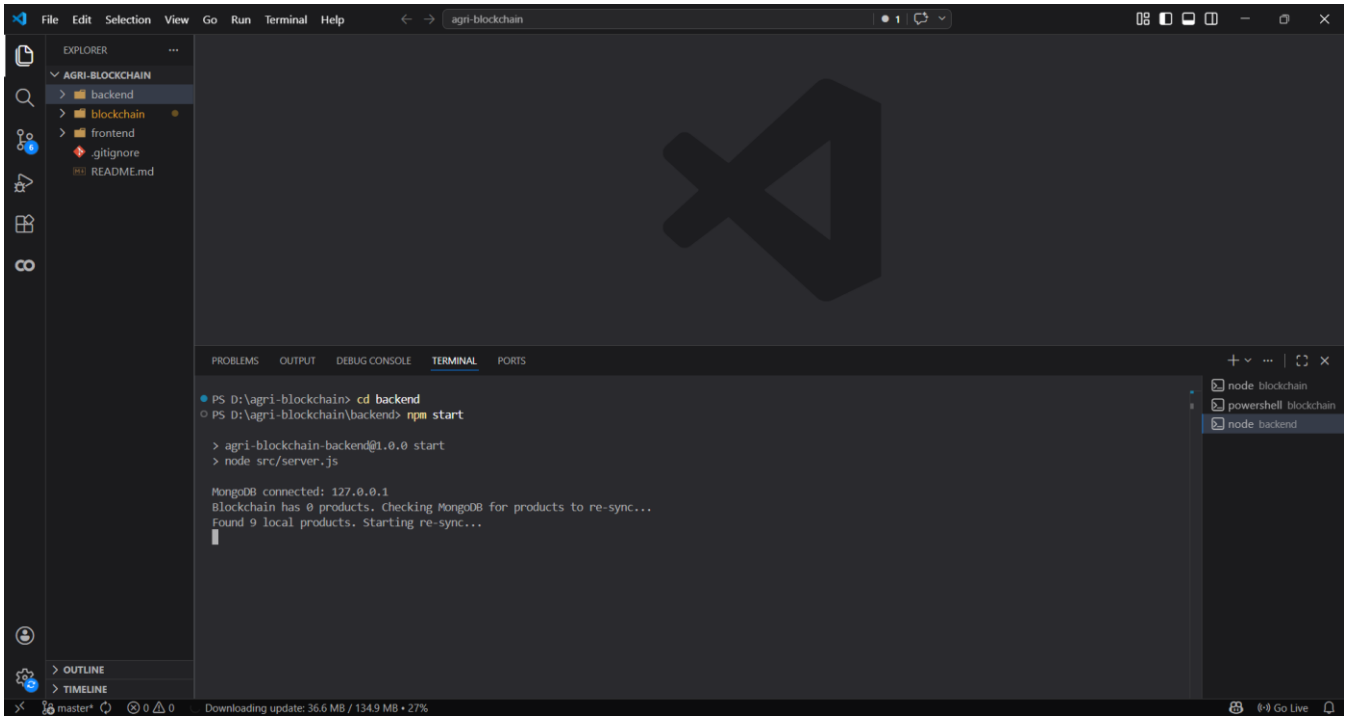
We Use to npm run ganache to run the block chain server



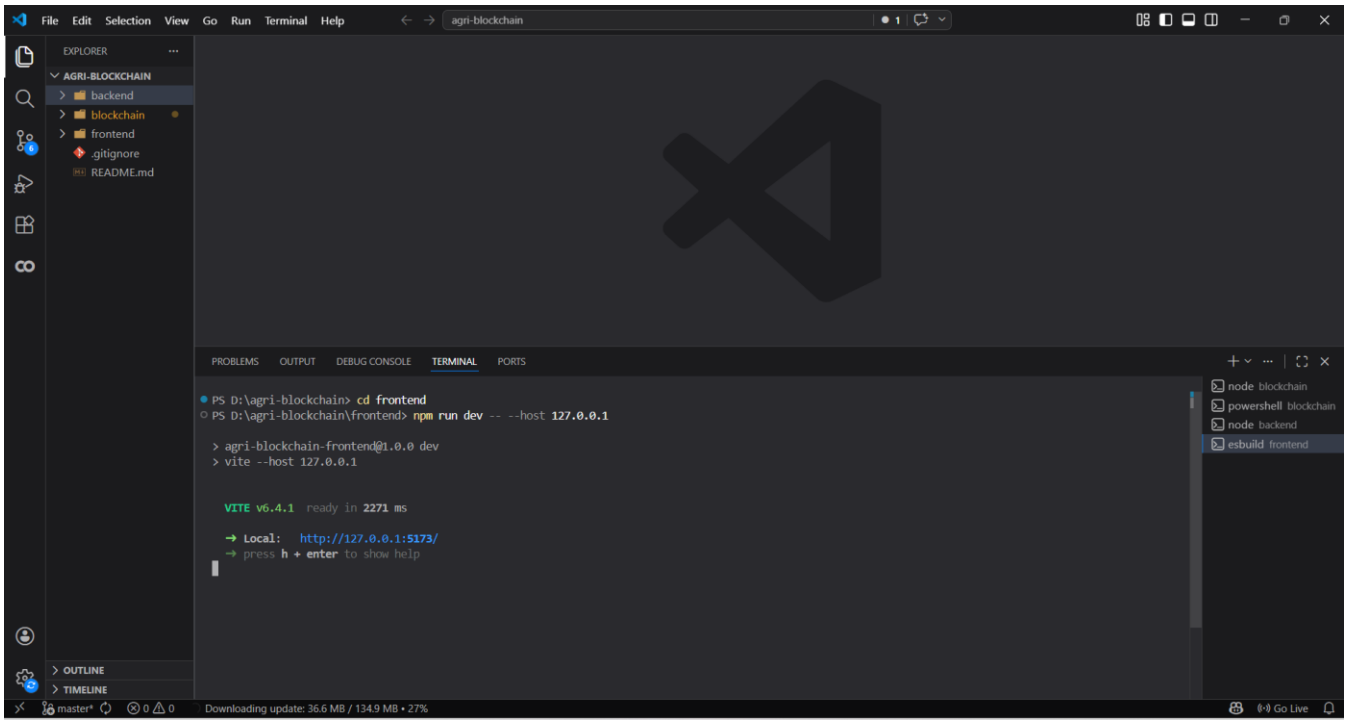
Then Reset the smart contracts using npm run reset



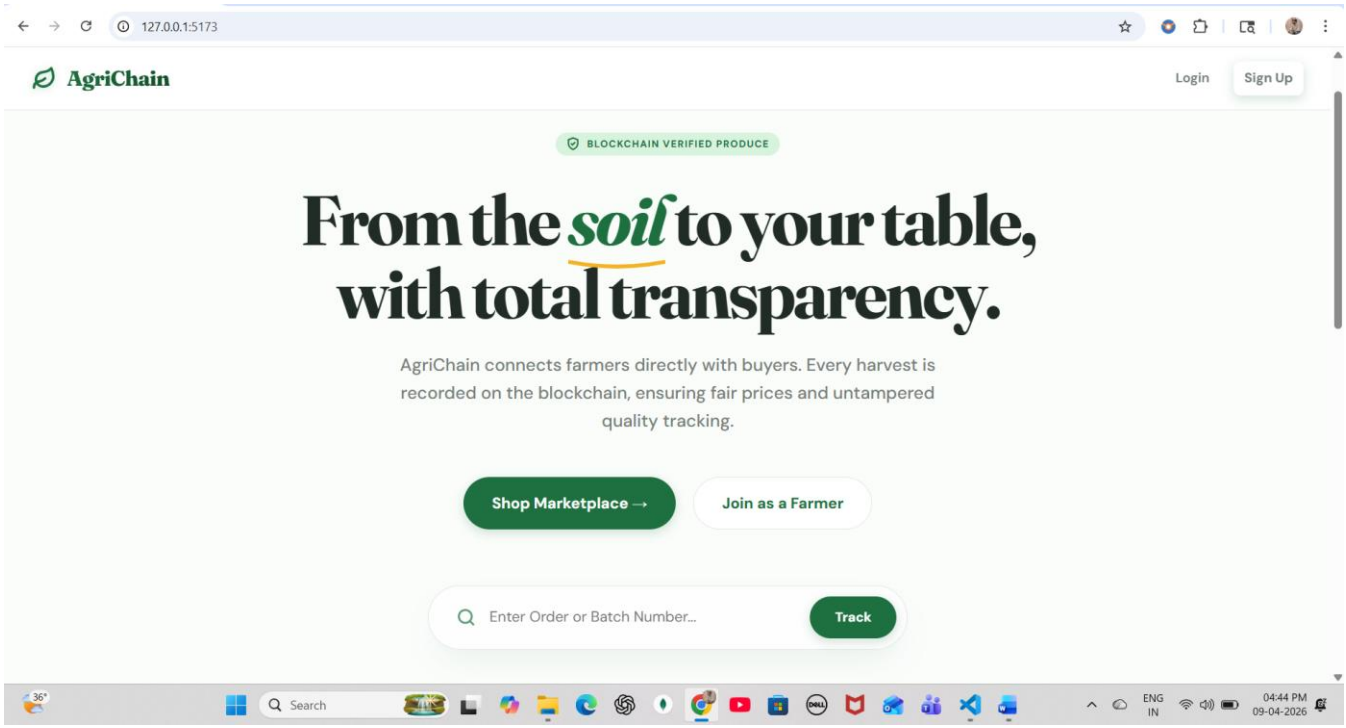
After that to start backend use npm start



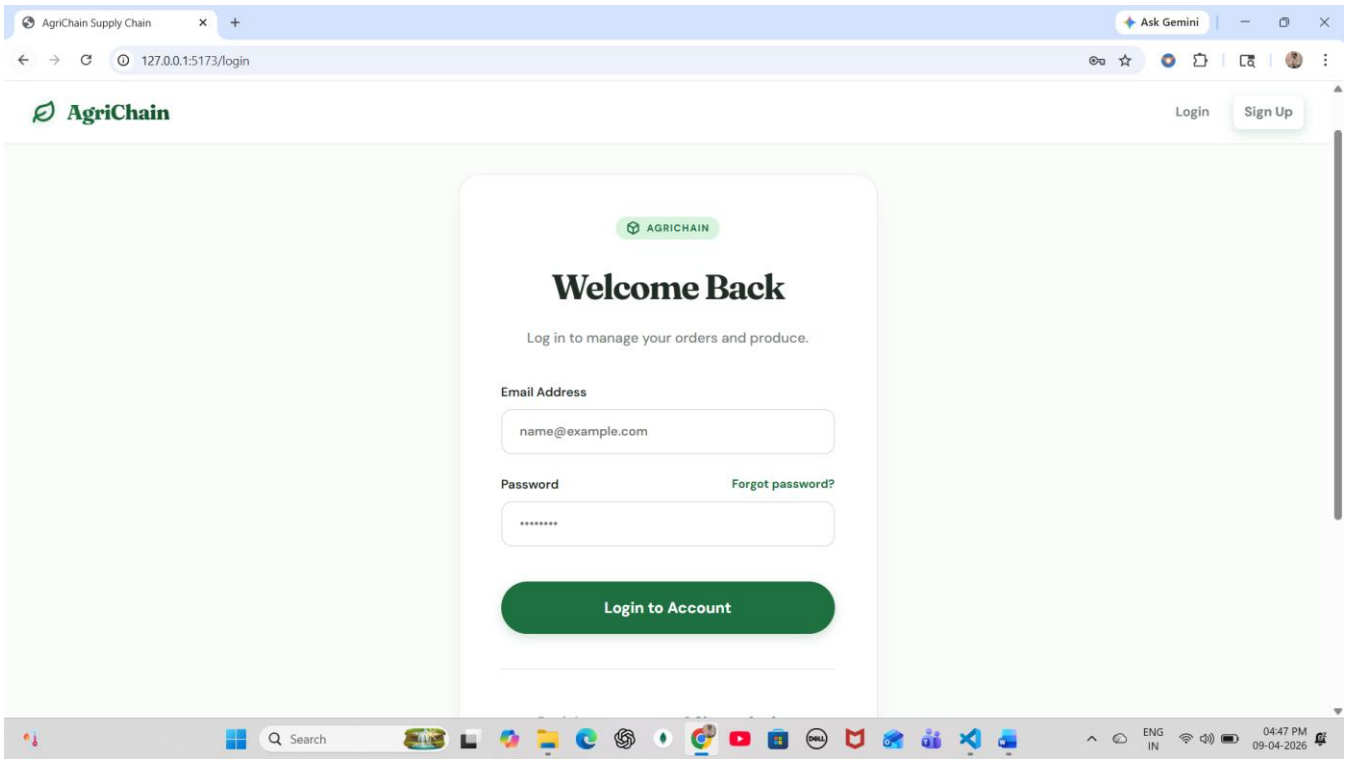
Then you start front end server using npm run dev



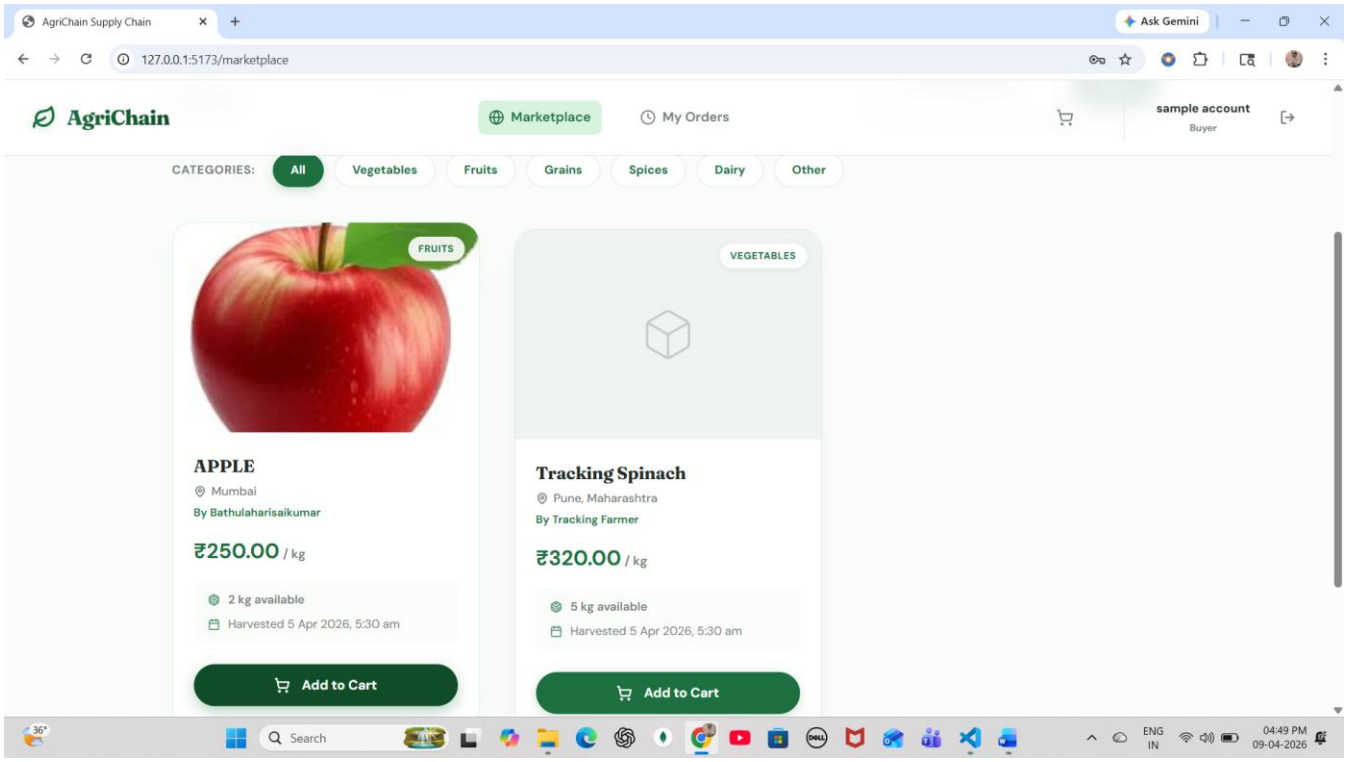
Click on the link to open the website to access



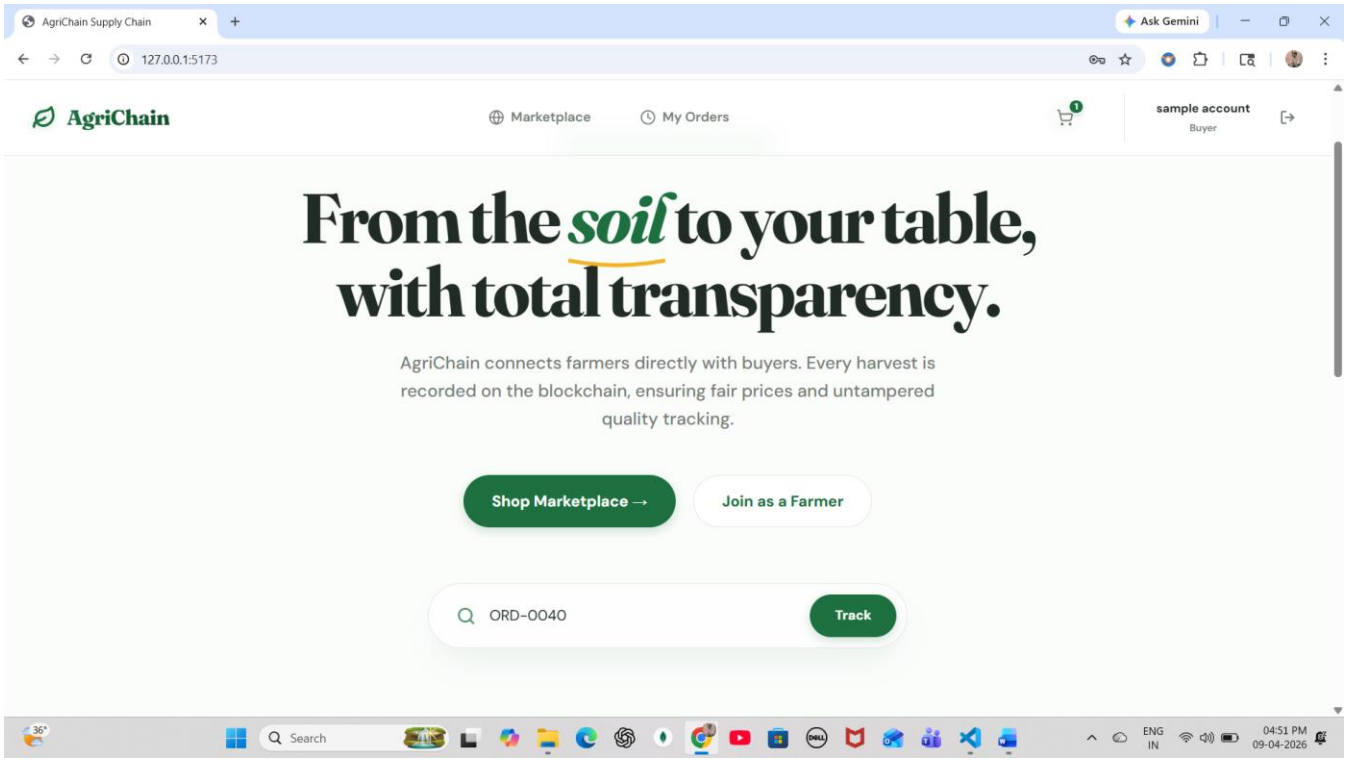
Open Login Account login as buyer or farmer account to continue



Click on market place add to products to cart check payment



If you want track order enter the order id and click on track to track your order



9.4 PERFORMANCE EVALUTION

METRIC	RESULT	TARGET
API Response Time (CRUD)	**< 200ms**	< 2000ms
Blockchain Tx Confirmation (Ganache)	**< 1 second**	< 5 seconds
JWT Token Validation	**< 5ms**	< 50ms
Page Load Time (React/Vite)	**< 1.5 seconds**	< 3 seconds
Order Number Generation	**< 50ms**	< 200ms
Razorpay Checkout Load	**< 2 seconds**	< 5 seconds
MongoDB Query (orders)	**< 100ms**	< 500ms

9.5 USER MANUAL

Step 1 — Start the Platform: Ensure Ganache is running on port 7545. Start MongoDB (mongod). Deploy smart contracts using truffle migrate --reset in /blockchain. Start the backend server using npm run dev in /server (port 5000). Start the frontend using npm run dev in /client (Vite serves on port 5173).

Step 2 — Register and Login: Navigate to http://localhost:5173. Click Register and select your role (Farmer/Buyer). Fill in name, email, and password. Login redirects to your role-specific dashboard automatically.

Step 3 — Farmer Product Listing: From Farmer Dashboard, click Add Product. Fill in product name, category, quantity, price per unit, harvest date, and origin. The system auto-generates a batch code. Submit to save in MongoDB and record on blockchain.

Step 4 — Buyer Order Placement: Browse the Marketplace, click a product, select quantity, click Add to Cart. Proceed to Checkout, enter delivery address, choose Online Payment (Razorpay) or COD, and confirm order. Note the Order Number displayed on confirmation.

Step 5 — Track Your Order: Navigate to Track Order (no login required). Enter the Order Number. View the complete status timeline and the Ethereum transaction hash as blockchain proof of each supply chain event.

Troubleshooting Guide

S.No	Problem	Solution
1	Server not starting	Install dependencies, check port and .env configuration
2	MongoDB connection error	Start MongoDB, verify connection string and port
3	Ganache / Blockchain issue	Run Ganache, check port 7545, redeploy contracts
4	Smart contract error	Compile using Truffle and migrate again
5	Payment (Razorpay) issue	Verify API keys and internet connection
6	Authentication error	Check JWT secret, token validity, and roles
7	API not responding	Verify backend server and API endpoints
8	Frontend not loading	Run frontend properly and check API URL

9	Order tracking issue	Verify order number and database records
10	General errors	Check logs, restart services, debug using console

9.6 CHALLANGES FACED AND THEIR SOLUTION

During the development of the **Farm-to-Market Supply Chain Platform**, several challenges were encountered and addressed effectively. One major challenge was integrating blockchain with the web application, which was resolved by using Web3.js in the backend to enable smooth communication between smart contracts and APIs. Maintaining data consistency between MongoDB and the blockchain was another issue, which was handled by storing transaction hashes and synchronizing records between both systems.

1. Blockchain Integration Complexity

Challenges:

• Integrating blockchain with a web application is complex due to communication between smart contracts and backend systems.

• Managing transaction signing and handling blockchain responses adds difficulty.

Solution:

• Used Web3.js to connect the backend with Ethereum smart contracts.

• Implemented backend-based transaction handling to simplify user interaction without requiring wallets.

2. Data Consistency Between Database and Blockchain

Challenges:

• Maintaining synchronization between MongoDB data and blockchain records is difficult.

• Any mismatch can lead to incorrect tracking or verification issues.

Solution:

• Stored blockchain transaction hashes in MongoDB for reference.

• Implemented synchronization logic to ensure both systems remain consistent.

3. Smart Contract Deployment Issues

Challenges:

• Errors during contract compilation and deployment.

• Managing contract addresses and network configurations.

Solution:

• Used Truffle framework for compilation and migration.

• Used Ganache for local blockchain testing and easy redeployment.

4. Payment Integration Challenges

Challenges:

• Handling secure online transactions and verification.

• Managing different payment methods like UPI and COD.

Solution:

• Integrated Razorpay test mode for online payments.

• Implemented payment verification before updating order status.

• Designed separate workflows for COD and online payments.

5. Real-Time Order Tracking Limitation

Challenges:

• Lack of real GPS hardware for real-time delivery tracking.

• Difficulty in demonstrating live tracking in a local environment.

Solution:

• Implemented GPS simulation to mimic real-time delivery movement.

• Updated location dynamically to demonstrate tracking functionality.

6. Security and Authentication Issues

Challenges:

• Ensuring secure user authentication and preventing unauthorized access.

• Protecting sensitive data such as user details and transactions.

Solution:

• Implemented JWT-based authentication and role-based access control.

• Used password hashing (bcrypt) and secure API handling.

• Stored sensitive data in environment variables for protection.

7. User Interface and Usability

Challenges:

• Designing an interface suitable for farmers with limited technical knowledge.

• Ensuring smooth navigation across different user roles.

Solution:

• Developed a simple and intuitive UI using React.js.

• Created separate dashboards for Buyer, Farmer, and Admin.

• Added clear navigation and tracking features for better usability.

9.7 ADDITIONAL TECHNICAL DETAILS

1. **Framework:** The project is developed using a full-stack architecture with **React.js** for the frontend and **Node.js with Express.js** for the backend. React enables dynamic and responsive user interfaces, while Express provides efficient API handling and server-side logic implementation.

2. **Database:** The system uses **MongoDB**, a NoSQL database, for storing application data such as users, products, orders, and transaction history. **Mongoose** is used as an Object Data Modeling (ODM) library to define schemas and manage database operations efficiently.

3. **Authentication and Security:** User authentication is implemented using **JWT (JSON Web Tokens)** to ensure secure access to protected routes. Passwords are encrypted using **bcrypt**, and role-based access control is applied to differentiate permissions for buyers, farmers, and administrators. Sensitive information such as API keys is stored in environment variables.

4. **API Development:** RESTful APIs are developed using Express.js to handle communication between frontend and backend. These APIs manage operations such as user authentication, product management, order processing, payment handling, and tracking. Proper validation and error handling mechanisms are implemented for reliability.

5. **Frontend Architecture:** The frontend is structured using reusable React components and pages. State management is handled using React Context API, enabling efficient data sharing across components. The UI is designed to be responsive and user-friendly, ensuring easy navigation for all types of users.

6. **Backend Architecture:** The backend follows a modular structure with separate folders for routes, controllers, models, middleware, and services. This separation improves code maintainability and scalability. Middleware is used for authentication, logging, and error handling.

7. **Blockchain Integration:** Blockchain functionality is implemented using **Ethereum smart contracts written in Solidity**. These contracts store important supply chain events such as product creation, order placement, payment updates, and delivery status. **Truffle** is used for contract compilation and deployment, while **Ganache** provides a local blockchain environment. **Web3.js** is used to interact with the blockchain from the backend.

8. **Routing Mechanism:** Frontend navigation is handled using **React Router**, enabling seamless page transitions. Backend routing is managed using Express.js, which maps API endpoints to appropriate controllers.

9. **Payment Integration:** The system integrates **Razorpay (test mode)** for online payment processing. Payment verification is performed securely before updating order status. Additionally, **Cash on Delivery (COD)** is supported as an alternative payment method.

10. **Tracking and Visualization:** Order tracking is implemented using a public tracking system where users can view order status and history. GPS simulation is used to demonstrate delivery tracking. Visualization tools like **Recharts** are used to display analytics and system data.

11. **Testing and Debugging:** The system is tested using tools like **Postman** for API testing and browser developer tools for frontend debugging. Manual testing is performed to ensure all workflows function correctly.

12. **Deployment:** The application is designed to run in a local development environment using Ganache for blockchain simulation. It can be deployed on cloud platforms such as **AWS, Heroku, or Vercel** for scalability. Environment variables are used to securely manage configuration settings.

13. **Performance Optimization:** Efficient data handling techniques are used to improve system performance. API responses are optimized, and unnecessary computations are minimized to ensure faster execution and better user experience.

CHAPTER 10

SYSTEM TESTING

System testing is a type of testing that verifies the entire software system as a whole, rather than individual software components or modules. It is performed after the completion of integration testing and ensures that all the components of the system are working correctly and meet the requirements specified in the system requirements specification.

System testing involves a series of tests that are performed on the system, including functional testing, performance testing, security testing, and usability testing. The primary goal of system testing is to identify defects or issues in the system and ensure that it meets the intended requirements.

Some of the key activities involved in system testing include:

1. Defining test scenarios and test cases based on the system requirements.
2. Executing tests on the system to validate that it meets the requirements.
3. Identifying and documenting defects or issues in the system and assigning them to the development team for resolution.
4. Conducting regression testing to ensure that any issues or defects found in previous test cycles have been resolved.
5. Conducting acceptance testing to ensure that the system is ready for release to end-users.

Overall, system testing is a critical component of the software development life cycle, and it is essential to ensure that the system meets the requirements and is functioning correctly before it is released to end-users.

10.1 TYPES OF TESTING

There are several types of software testing that are typically performed to ensure the quality and functionality of software products. Here are some of the most common types of testing:

10.1.1 Unit testing:

This is the testing of individual components or units of code to ensure that they function correctly in isolation. Unit testing is typically performed by developers and is an important part of the development process.

Unit testing is a type of software testing in which individual units or components of the software are tested in isolation from the rest of the system. The purpose of unit testing is to validate that each unit or component of the software performs as expected and meets its specified requirements.

During unit testing, a developer writes test cases for each unit and executes them in isolation from the rest of the software. The tests are typically automated and are designed to verify the behavior of the unit under various input conditions and scenarios. The tests may include positive and negative scenarios, boundary conditions, and error conditions.

Unit testing is important because it allows developers to identify and fix defects early in the development process, before the code is integrated with the rest of the system. This reduces the cost and effort of finding and fixing defects later in the development cycle. Unit testing also provides a level of confidence that the individual units or components of the software are functioning correctly, which can help to reduce the risk of defects in the overall system.

Unit testing is typically performed using specialized tools and frameworks, such as JUnit for Java or Unknit for .NET. These tools provide a framework for writing and executing tests, and may also provide features such as test reporting and code coverage analysis.

Overall, unit testing is an important practice in software development that helps to improve the quality of the software and reduce the risk of defects.

10.1.2 Integration testing: This type of testing checks the interactions between different components of the software system to ensure that they work together as expected. Integration testing is often performed after unit testing and before system testing.

Integration testing is a software testing technique that verifies the interaction between different software modules or components. It is performed after the completion of unit testing and before the system testing phase. The objective of integration testing is to identify defects in the interfaces between the software components, as well as issues related to the interaction between these components.

10.1.3 There are several types of integration testing, including:

Big Bang Integration Testing: This approach involves integrating all the components of the software system in one go and testing them as a whole.

Top-Down Integration Testing: In this approach, the higher-level modules are tested first, and then the lower-level modules are gradually integrated and tested.

Bottom-Up Integration Testing: In this approach, the lower-level modules are tested first, and then the higher-level modules are integrated and tested.

Sandwich Integration Testing: This approach involves a combination of top-down and bottom-up integration testing, where the modules are tested in both directions.

Hybrid Integration Testing: This approach combines two or more of the above-mentioned approaches to suit the specific requirements of the software project.

The main benefits of integration testing include the early detection and resolution of defects in the software components, the identification of design flaws and architectural issues, and the assurance that the different components of the software system work together seamlessly.

10.1.4 System testing:

This is the testing of the entire system as a whole to ensure that all components work together and the system meets its functional and non-functional requirements. System testing is typically performed by a separate testing team.

System testing is a level of testing that involves testing the complete and integrated system against the

specified requirements. It tests the system as a whole, rather than individual components. The purpose of system testing is to ensure that all the components of the system work together as intended and that the system meets its functional and non-functional requirements.

There are several types of system testing, including:

1. Functional testing: This type of testing focuses on testing the system's functionality against the

requirements specification. The goal is to ensure that the system performs all the functions that it is supposed to perform.

2. Performance testing: This type of testing is used to measure the system's performance against specified parameters, such as response time, throughput, and scalability. The goal is to ensure that the system can handle the expected load and perform adequately.

3. Load testing: This type of testing is used to measure the system's performance under heavy loads, such as high user traffic. The goal is to ensure that the system can handle the expected load without crashing or slowing down.

4. Stress testing: This type of testing is used to test the system's behaviour under extreme conditions, such as very high loads or limited system resources. The goal is to identify the system's breaking point and ensure that it can recover gracefully.

5. Usability testing: This type of testing is used to evaluate the system's ease of use and user interface. The goal is to ensure that the system is user-friendly and that users can complete their tasks without difficulty.

6. Security testing: This type of testing is used to evaluate the system's security features and identify any vulnerabilities. The goal is to ensure that the system is secure and protected against unauthorized access or malicious attacks.

Acceptance testing: This type of testing is performed to ensure that the software meets the requirements and expectations of the customer or end user. Acceptance testing can be performed by the customer or end user, or by a separate testing team.

Acceptance testing is a type of testing that evaluates whether a system or application meets the requirements and specifications of the stakeholders or end-users. It is a critical phase of software testing that is conducted after the completion of functional and system testing.

The purpose of acceptance testing is to ensure that the system or application is ready for deployment and meets the needs of the end-users. It helps to identify any defects or issues before the final release and ensures that the product is of high quality.

Acceptance testing can be divided into two main types:

User acceptance testing (UAT): In this type of testing, the end-users or stakeholders of the system or application perform testing to ensure that the system meets their requirements and specifications. UAT is typically conducted in a real-world environment to ensure that the system can perform as expected in a production environment.

Business acceptance testing (BAT): In this type of testing, the business team or product owner performs testing to ensure that the system meets the business requirements and goals. BAT focuses on testing the system from a business perspective and ensures that the system can support the business processes and workflows.

Acceptance testing can be performed manually or through automation tools, depending on the complexity and size of the system or application. The testing team should create detailed test cases and scenarios to cover all the requirements and ensure that the system meets the expected outcomes.

Regression testing: This is the testing of previously tested software after changes have been made to ensure that the changes have not introduced new defects or issues. Regression testing is important to ensure that the software remains stable and functional throughout the development process.

Regression testing is a type of testing that is performed to ensure that changes or modifications made to an existing software system have not affected its existing functionality. It involves retesting the unchanged parts of the system along with the modified parts to ensure that the system works as expected. The goal of regression testing is to catch any defects or errors that may have been introduced during the modification process.

Regression testing can be performed manually or through automated testing tools. Some common techniques used for regression testing include retesting, test case prioritization, and selecting a subset of test cases that are most likely to be affected by the changes.

There are several benefits of regression testing. It helps to identify and fix defects early on in the development process, which can save time and money in the long run. It also helps to ensure that the system continues to function as intended even after modifications or changes have been made.

- **Performance testing:** This type of testing checks the performance and responsiveness of the software under various conditions, such as high user traffic or heavy data loads. Performance testing is important to ensure that the software can handle expected usage levels.

Performance testing is a type of testing that is used to evaluate how well a system performs under specific conditions, such as high traffic or heavy load. The goal of performance testing is to identify and resolve performance bottlenecks that may cause the system to slow down, crash, or otherwise fail to meet its performance goals.

There are several types of performance testing, including:

- **Load testing**: This type of testing involves simulating a high volume of users accessing the system simultaneously to see how the system handles the load. The goal is to identify the maximum load that the system can handle before it begins to slow down or fail.

- **Stress testing**: This type of testing involves pushing the system beyond its normal capacity to see how it performs under extreme conditions. The goal is to identify the breaking point of the system and understand how it behaves when it reaches that point.

- **Endurance testing**: This type of testing involves evaluating the system's performance over an extended period of time to see how it performs under sustained load. The goal is to identify any performance degradation that may occur over time.

- **Spike testing:** This type of testing involves simulating sudden spikes in user traffic to see how the system responds. The goal is to identify how the system handles sudden changes in load and whether it can recover quickly.

- S**calability testing:** This type of testing involves evaluating how well the system can scale up or down to handle changes in user traffic or load. The goal is to identify any limitations in the system's ability to scale and to determine how many users or transactions it can handle before performance is affected.

Performance testing is an important part of software development because it helps ensure that the

system can handle the expected workload and perform well under all conditions. It can also help identify potential issues before they become critical problems, allowing developers to make changes and improvements to the system as needed.

- **Security testing:** This type of testing checks the security of the software to ensure that it is protected against unauthorized access, attacks, and vulnerabilities. Security testing is important to ensure the safety and confidentiality of user data.

Security testing is a type of testing that checks the system or application for vulnerabilities, threats, and risks that could compromise its confidentiality, integrity, and availability. It involves testing the system or application to identify and address security-related issues such as unauthorized access, data theft, data loss, and data corruption.

- **There are several types of security testing, including:**
    - **Vulnerability scanning**: This type of testing involves using automated tools to scan the system or application for known vulnerabilities such as unpatched software, weak passwords, and misconfigured settings.
    - **Penetration testing**: This involves simulating an attack on the system or application to identify weaknesses that could be exploited by attackers. It can be either manual or automated.
    - **Security auditing**: This involves reviewing the system or application's design and architecture to identify potential security issues.
    - **Security configuration testing:** This type of testing checks the configuration settings of the system or application to ensure that they are in compliance with industry standards and best practices.
    - **Access control testing:** This type of testing checks the system or application's access controls to ensure that only authorized users have access to sensitive data and functions.

Overall, security testing is critical to ensure that the system or application is secure from cyber threats and meets regulatory compliance requirements.

These are just a few examples of the many types of testing that can be performed to ensure the quality and functionality of software products. Different types of testing may be more or less relevant depending on the specific software being developed and the needs and requirements of the stakeholders involved.

10.2 SYSTEM TESTING

System testing is a type of software testing that evaluates the complete software system as a whole, rather than testing individual software components or units. It involves testing the system as a whole to ensure that all of its components work together seamlessly and meet the specified requirements.

System testing is typically performed after unit testing and integration testing have been completed. It is a critical step in the software development process, as it verifies that the system is functioning correctly and meets the needs of the end-users.

There are several types of system testing, including:

- **Functional testing**: This type of testing checks the functionality of the software system to ensure that it performs as expected and meets the specified requirements.
- **Performance testing:** This type of testing checks the system's performance under various load conditions, such as high user traffic or data processing. It helps identify performance bottlenecks and ensure the system can handle the expected workload.
- **Compatibility testing:** This type of testing checks the system's compatibility with different software, hardware, and network configurations to ensure that it works on various platforms and environments.
- **Usability testing:** This type of testing checks the system's user-friendliness and ease of use to ensure that it meets the needs of the end-users.
- **Security testing:** This type of testing checks the system's security features to ensure that it is protected against potential threats such as hacking, unauthorized access, and data breaches.

Overall, system testing is essential to ensure that the software system is fully functional, secure, and meets the needs of the end-users. It helps identify any issues or defects before the system is released to production, saving time and resources in the long run.

CHAPTER 11

CONCLUSION

The **Farm-to-Market Supply Chain Platform** successfully demonstrates how blockchain technology can be integrated with modern web applications to improve transparency, traceability, and efficiency in agricultural supply chains. The system establishes a direct connection between farmers and buyers, eliminating intermediaries and enabling farmers to receive a greater share of the final market price.

The platform is developed using a robust technology stack that includes React.js for the frontend, Node.js and Express.js for backend processing, MongoDB for data storage, and Ethereum smart contracts for secure and immutable record keeping. By recording key supply chain events such as product creation, order placement, payment updates, and delivery status on the blockchain, the system ensures data integrity and builds trust among users.

The implementation of role-based access control allows different users such as buyers, farmers, and administrators to interact with the system efficiently. Features like real-time order tracking, GPS simulation, and public traceability enhance user experience and provide complete visibility into the product lifecycle. Additionally, secure payment integration using Razorpay and support for Cash on Delivery make the system flexible and user-friendly.

The project also demonstrates a hybrid architecture where MongoDB handles operational data while blockchain is used for verification and proof storage. This approach balances performance with security and makes the system suitable for both academic demonstration and potential real-world applications.

Overall, the system meets its objectives by providing a scalable, secure, and transparent solution for agricultural marketplaces. It lays a strong foundation for future enhancements such as cloud deployment, mobile applications, real-time GPS integration, advanced analytics, and integration with external logistics services, making it a valuable contribution to blockchain-based supply chain management.

CHAPTER 12

FUTURE SCOPE

The Farm-to-Market Supply Chain Platform has significant potential for future enhancements and real-world deployment. While the current system is designed for academic and prototype purposes, it can be extended to support advanced features and large-scale applications.

One of the major future improvements is deployment on a public blockchain network such as Ethereum testnets or mainnet, which would provide real-world decentralization and stronger trust. The system can also be enhanced by integrating mobile applications (Android/iOS) to make the platform more accessible to farmers and buyers.

Another important extension is the implementation of real-time GPS tracking using IoT devices, replacing the current GPS simulation. This would allow accurate monitoring of product movement during delivery. Additionally, QR code-based product verification can be introduced, enabling users to scan and view complete product history and blockchain records instantly.

The platform can further be improved by integrating advanced analytics and AI-based features, such as demand prediction, price forecasting, and recommendation systems. This will help farmers make better decisions regarding crop sales and pricing.

Future development can also include integration with government agricultural systems and digital marketplaces, ensuring compliance with regulations and wider adoption. Features like multi-language support can be added to improve usability for farmers from different regions.

From a technical perspective, the system can be scaled using cloud deployment (AWS, Azure, GCP) and enhanced with real-time communication technologies like WebSockets for faster updates. Security can be strengthened with advanced encryption techniques and multi-factor authentication.

Overall, the project has strong potential to evolve into a fully functional, scalable, and production-ready agricultural marketplace that leverages blockchain technology for transparency, trust, and efficiency.

CHAPTER 13

REFERENCES

[1] **Feng Tian (2016):** "An Agri-food Supply Chain Traceability System for China Based on RFID & Blockchain Technology." 2016 13th International Conference on Service Systems and Service Management (ICSSSM), IEEE.

[2] **Caro et al. (2018):** "AgriBlockIoT: A Novel Agri-food Supply Chain Solution Using Blockchain and Internet of Things." International Conference on Future Networks and Distributed Systems (ICFNDS), ACM.

[3] **Kamilaris, A., Fonts, A., & Prenafeta-Bold'u, F.X. (2019):** "The Rise of Blockchain Technology in Agriculture and Food Supply Chains." Trends in Food Science & Technology, Elsevier, Vol. 91, pp. 640-652.

[4] **Zhao et al. (2019):** "A Review on Blockchain Applications in Agri-food Value Chain." Transactions of the Chinese Society of Agricultural Engineering (CSAE).

[5] **Shahid, A., et al. (2020):** "Blockchain-Based Agri-Food Supply Chain: A Complete Solution." IEEE Access, Vol. 8, pp. 69230-69243.

[6] **Marchese, A., & Tomarchio, O. (2022):** "A Blockchain-Based Traceability Management System for Agri-Food Products." IEEE Blockchain Technical Briefs.

[7] **Yogarajan, G., et al. (2023):** "A Systematic Review on Blockchain Adoption in Agri-Food Supply Chain." Computers & Industrial Engineering, Elsevier.

[8] **Rashid, A., et al. (2024):** "Blockchain-Enabled Web Application for Fair Pricing and Traceability in Agriculture." IEEE Transactions on Agricultural & Environmental Engineering.

[9] **Buterin, V. (2014):** "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform." Ethereum White Paper.

[10] **Wood, G. (2014):** "Ethereum: A Secure Decentralised Generalised Transaction Ledger." Ethereum Yellow Paper.

[11] **Nakamoto, S. (2008):** "Bitcoin: A Peer-to-Peer Electronic Cash System." Bitcoin White Paper.

[12] **Truffle Suite Documentation:** "Truffle: The Most Popular Development Framework for Ethereum." trufflesuite.com.

[13] **Razorpay Documentation:** "Razorpay Payment Gateway Integration Guide for Node.js." razorpay.com/docs.

[14] **MongoDB Documentation:** "MongoDB Manual — Aggregation, Indexing, and Schema Design." mongodb.com/docs.

APPENDICES

The appendices of a project typically include additional materials and information that support the main content of the project. Some examples of items that may be included in the appendices are:

APPENDIX A: FRONTEND MODULES

The frontend of the system is developed using React.js and provides an interactive user interface. It includes modules such as login, registration, marketplace, cart, dashboards for different users, and order tracking. The interface is designed to be responsive and user-friendly, enabling smooth navigation for buyers, farmers, and administrators.

APPENDIX B: BACKEND MODULES

The backend is developed using Node.js and Express.js. It handles API requests, business logic, authentication, and communication between the frontend, database, and blockchain. The backend ensures secure data processing and implements core functionalities such as product management, order processing, and tracking.

APPENDIX C: DATABASE STRUCTURE

The system uses MongoDB to store data such as users, products, and orders. The database is designed using collections with structured fields to ensure efficient data storage and retrieval. Relationships between data are maintained using unique identifiers.

APPENDIX D: BLOCKCHAIN MODULE

The blockchain module is implemented using Ethereum smart contracts written in Solidity. It records important events such as product creation, order placement, payment updates, and delivery status. This ensures transparency, immutability, and trust in the system.

APPENDIX E: PAYMENT MODULE

The system integrates Razorpay (test mode) for handling online payments and also supports Cash on Delivery (COD). Payment verification is performed before updating order status to ensure secure transactions.

APPENDIX F: ORDER LIFECYCLE

The order lifecycle follows multiple stages including pending, confirmed, shipped, out for delivery, and delivered. Each stage is tracked and recorded to provide complete visibility of the order process.

APPENDIX G: SECURITY MECHANISMS

The system uses JWT-based authentication and role-based access control to secure user data and system operations. Passwords are encrypted, and sensitive data is protected using environment variables.

APPENDIX H: SYSTEM WORKFLOW

The system workflow includes user interaction through the frontend, processing via backend APIs, data storage in MongoDB, and recording of key events on blockchain. This ensures a seamless and transparent operation of the platform.